Composable Execution Systems in Go

A handbook for small semantic kernels, in-process flow execution, and durable artifact orchestration

Reference architecture and implementation handbook

August 2026

Contents

1 Abstract **6**

2 Preface **6**
2.1 Purpose . . . 6
2.2 Source basis . . . 6
2.3 Validation status . . . 7
2.4 Reading routes . . . 7

3 1. The problem is two problems **7**
3.1 1.1 Fine-grained execution . . . 7
3.2 1.2 Durable orchestration . . . 8
3.3 1.3 The layer boundary . . . 8
3.4 1.4 Why not one universal workflow abstraction? . . . 8

4 2. Semantic foundations **10**
4.1 2.1 Information, views, and traces . . . 10
4.1.1 Semantic information . . . 10
4.1.2 View . . . 10
4.1.3 Trace . . . 10
4.2 2.2 Three identities . . . 10
4.3 2.3 Content-addressed references . . . 10
4.4 2.4 Partial functions and semantic caches . . . 12
4.5 2.5 Checked union and ACI laws . . . 12
4.6 2.6 Pure functions remain ordinary functions . . . 13

5 3. A method for small extensible kernels **13**
5.1 3.1 Begin with observation levels . . . 13
5.2 3.2 Put laws before fields . . . 13
5.3 3.3 Prefer algebraic values to manager objects . . . 14
5.4 3.4 Use narrow graft points . . . 14
5.5 3.5 Keep optional algebra optional . . . 14

6 4. `flow`: mandate and non-goals **14**
6.1 4.1 Contract . . . 14
6.2 4.2 Why the old `Step` shape was rejected . . . 15
6.3 4.3 Kernel inventory . . . 15

7 5. `flow` semantic API **15**
7.1 5.1 `Key` . . . 15
7.2 5.2 Canonicalization and versioning . . . 16
7.3 5.3 `Call[I]` . . . 17

7.4 5.4 `Processor[I,O]` . . . 17
7.5 5.5 `Outcome` and error levels . . . 17

8 6. `flow.Run` operational semantics 18
8.1 6.1 Signature . . . 18
8.2 6.2 Execution phases . . . 18
8.2.1 Phase 1: validate and group . . . 19
8.2.2 Phase 2: cache lookup . . . 19
8.2.3 Phase 3: form a retry wave . . . 19
8.2.4 Phase 4: execute batches concurrently . . . 19
8.2.5 Phase 5: commit successes immediately . . . 19
8.2.6 Phase 6: classify failures . . . 19
8.2.7 Phase 7: wait once for the next wave . . . 19
8.2.8 Phase 8: expand outcomes . . . 19
8.3 6.3 Stable order under concurrency . . . 20
8.4 6.4 Duplicate transparency . . . 20
8.5 6.5 Partial retry . . . 20
8.6 6.6 Cache transparency . . . 20
8.7 6.7 Report semantics . . . 20

9 7. `flow` graft points 21
9.1 7.1 Cache . . . 21
9.2 7.2 Codec . . . 21
9.3 7.3 Retry classifier . . . 21
9.4 7.4 Admission . . . 21
9.5 7.5 Observer . . . 22
9.6 7.6 Clock . . . 22
9.7 7.7 Explicit assembly . . . 22

10 8. `flow` laws and tests 22
10.1 8.1 Operational invariance . . . 22
10.2 8.2 Permutation qualification . . . 23
10.3 8.3 Retry transparency . . . 23
10.4 8.4 Budget accounting . . . 23
10.5 8.5 Cache functionality . . . 23
10.6 8.6 Alignment custody . . . 23
10.7 8.7 Race tests . . . 23
10.8 8.8 Example: embedding 30,000 representations . . . 23
10.9 8.9 Migration from a broad step API . . . 25

11 9. `ragjobs`: mandate and architecture 25
11.1 9.1 Contract . . . 25
11.2 9.2 Durable job granularity . . . 25
11.3 9.3 Reference indexing plan . . . 26
11.4 9.4 Mutable control plane, immutable data plane . . . 26
11.4.1 Mutable control plane . . . 26
11.4.2 Immutable data plane . . . 27

12 10. References, artifacts, and semantic outputs 27
12.1 10.1 `Ref` . . . 27
12.2 10.2 `Artifact` . . . 27
12.3 10.3 Named output maps . . . 27
12.4 10.4 Artifact immutability law . . . 28

13 11. Plans as canonical durable values 28
13.1 11.1 `NodeSpec` . . . 28
13.1.1 Semantic protocol . . . 28
13.1.2 Causality . . . 28

13.1.3 Durable operational policy . . . 28
13.2 11.2 Handler protocol versioning . . . 29
13.3 11.3 Ports . . . 29
13.4 11.4 `Finalize` . . . 29
13.5 11.5 Canonical plan identity . . . 29
13.6 11.6 Direct plan data versus composition helpers . . . 30
13.7 11.7 Composition laws . . . 30
13.7.1 Sequential associativity . . . 30
13.7.2 Identity . . . 30
13.7.3 Parallel associativity and symmetry . . . 30

14 12. Semantic invocation identity **30**
14.1 12.1 Run input . . . 30
14.2 12.2 Node semantic key . . . 31
14.3 12.3 Why direct dependency digests matter . . . 31
14.4 12.4 Plan identity versus semantic key . . . 32

15 13. Operational state machine **32**
15.1 13.1 States . . . 32
15.2 13.2 The Store is the semantics . . . 32
15.3 13.3 Configuration model . . . 33
15.4 13.4 CREATE . . . 33
15.5 13.5 Readiness . . . 34
15.6 13.6 CLAIM . . . 34
15.7 13.7 Lease as capability . . . 34
15.8 13.8 HEARTBEAT . . . 35
15.9 13.9 COMPLETE . . . 35
15.10 13.10 FAIL and RETRY . . . 35
15.11 13.11 RECOVER . . . 36
15.12 13.12 CANCEL . . . 36
15.13 13.13 FINALIZE . . . 36

16 14. Safety properties **36**
16.1 14.1 Dependency safety . . . 36
16.2 14.2 Stale-worker exclusion . . . 36
16.3 14.3 Terminal monotonicity . . . 36
16.4 14.4 Single accepted outcome per attempt . . . 37
16.5 14.5 Event-prefix integrity . . . 37
16.6 14.6 Cache functionality . . . 37
16.7 14.7 Conservative budget safety . . . 37
16.8 14.8 Concurrency-key exclusion . . . 37
16.9 14.9 Eventual terminality assumptions . . . 38

17 15. Worker and handler protocol **38**
17.1 15.1 Handler . . . 38
17.2 15.2 Registry . . . 38
17.3 15.3 Worker interpreter . . . 38
17.4 15.4 Cache reuse . . . 39
17.5 15.5 Parent shutdown . . . 39
17.6 15.6 Panics . . . 39

18 16. Store implementations **39**
18.1 16.1 Memory Store . . . 39
18.2 16.2 Exported reference state . . . 39
18.3 16.3 File Store . . . 40
18.4 16.4 Store conformance . . . 40
18.5 16.5 PostgreSQL refinement obligation . . . 40

18.616.6 River refinement obligation 41

19 17. Immutable artifacts and publication 41
19.117.1 Content-addressed artifact store 41
19.217.2 Why artifacts, not directories in progress 41
19.317.3 Publication alias 41
19.417.4 Compare-and-swap semantics 42
19.517.5 Publication is not cacheable 42
19.617.6 Rollback 42
19.717.7 Garbage collection 42

20 18. Change capture and triggers 43
20.118.1 The dual-write problem 43
20.218.2 Source revision 43
20.318.3 Normalized changes 43
20.418.4 Contiguous adoption 43
20.518.5 Snapshot consistency 44
20.618.6 Configuration changes 44

21 19. Evaluation, quality gates, and optimization 44
21.119.1 Evaluation as artifacts 44
21.219.2 Deterministic fan-out/fan-in 44
21.319.3 Stochastic judges 45
21.419.4 Quality gates 45
21.519.5 `ragopt` as a workload, not a scheduler 45

22 20. Composing `flow` inside `ragjobs` 46
22.120.1 The pattern 46
22.220.2 Nested retry discipline 46
22.320.3 Failure mapping 47
22.420.4 Budget mapping 47
22.520.5 Trace linkage 47

23 21. Extensibility without a framework monoculture 47
23.121.1 Graft-point map 47
23.221.2 Plugin means package plus interface 49
23.321.3 Capability extension versus kernel expansion 49
23.421.4 Small interfaces and capability discovery 49
23.521.5 Version every persisted protocol 49

24 22. Designing elegant semantically sound APIs 50
24.122.1 State the equivalence first 50
24.222.2 Separate values from computations 50
24.322.3 Separate denotation from interpretation 50
24.422.4 Prefer data plus interpreter 50
24.522.5 Reject impossible ambiguity early 51
24.622.6 Do not overclaim algebra 51
24.722.7 Make partial failure explicit 51
24.822.8 Keep the happy path ordinary 51

25 23. Worked durable indexing example 52
25.123.1 Plan 52
25.223.2 Handler behavior 52
25.323.3 Publication 52
25.423.4 Observed result 52

26 24. Migration for TTC, GEC, ragkit, and ragopt 53
26.124.1 Migration principle 53

26.2 24.2 Phase 0: identity hardening . . . 53
26.3 24.3 Phase 1: introduce shared references . . . 53
26.4 24.4 Phase 2: migrate local execution . . . 53
26.5 24.5 Phase 3: extract coarse services . . . 53
26.6 24.6 Phase 4: shadow durable runs . . . 54
26.7 24.7 Phase 5: manual publication . . . 54
26.8 24.8 Phase 6: change capture . . . 54
26.9 24.9 Phase 7: automatic gated promotion . . . 54
26.10 24.10 `ragopt` migration . . . 54

27 25. Verification strategy 55
27.1 25.1 Test the laws, not only examples . . . 55
27.2 25.2 Differential Store testing . . . 55
27.3 25.3 Crash injection . . . 55
27.4 25.4 Race and schedule tests . . . 56
27.5 25.5 Property generation . . . 56
27.6 25.6 Production adapter test matrix . . . 56

28 26. Operations and observability 56
28.1 26.1 Durable events versus telemetry . . . 56
28.2 26.2 Metrics . . . 57
28.3 26.3 Tracing . . . 57
28.4 26.4 Administrative actions . . . 57
28.5 26.5 Incident interpretation . . . 57
28.5.1 Semantic cache conflict . . . 57
28.5.2 Stuck running node . . . 57
28.5.3 Outbox gap . . . 57
28.5.4 Gate rejection . . . 58
28.5.5 Bad active artifact . . . 58

29 27. Security and multi-tenant concerns 58
29.1 27.1 Secrets . . . 58
29.2 27.2 Authorization projection . . . 58
29.3 27.3 Immutable artifacts and deletion . . . 58
29.4 27.4 Supply-chain identity . . . 58

30 28. Performance and scaling 58
30.1 28.1 Scale the right layer . . . 58
30.2 28.2 Queue decomposition . . . 59
30.3 28.3 Backpressure . . . 59
30.4 28.4 Embedding critical path . . . 59
30.5 28.5 Whole build path . . . 59

31 29. Limitations and future extensions 60
31.1 29.1 No live PostgreSQL interpreter in this version . . . 60
31.2 29.2 No compiled River adapter . . . 60
31.3 29.3 Nominal ports . . . 60
31.4 29.4 Static plans . . . 60
31.5 29.5 Partial rerun . . . 60
31.6 29.6 Richer resource grades . . . 60
31.7 29.7 Segment-level incremental indexing . . . 60
31.8 29.8 Formal mechanization . . . 61

32 30. Recommended engineering discipline 61

33 Appendix A. Package map 61
33.1 A.1 `flow` . . . 61
33.2 A.2 `ragjobs` . . . 61

33.3A.3 Integration example 62

34 Appendix B. Transition table 62

35 Appendix C. Law checklist 63
35.1 C.1 `flow` 63
35.2 C.2 `ragjobs` 63

36 Appendix D. Production readiness checklist 64

37 Appendix E. Glossary 64

38 Appendix F. Prior work and implementation evidence 65

39 Closing perspective 65

1 Abstract

Production retrieval-augmented generation systems require two forms of execution that are often confused. Fine-grained provider work must be batched, rate-limited, retried, cached, and collected in a stable order inside one process. Coarse indexing, evaluation, and publication work must survive process death, deployment, duplicate delivery, stale workers, and source changes. One abstraction should not own both problems.

This handbook develops and documents two standalone Go modules:

- **`flow`**, a small in-process interpreter for semantically identified requests;
- **`ragjobs`**, a transport-independent durable orchestration kernel for coarse artifact-producing computations.

The design starts from explicit semantics rather than compatibility with an existing API. Pure functions remain ordinary Go. Framework machinery appears only at effect, resource, persistence, and publication boundaries. The common vocabulary is deliberately small: semantic references, immutable artifacts, deterministic requests, checked accumulation, deterministic views, attempts, traces, and atomic state transitions.

The handbook combines mathematical foundations, API design method, implementation walkthroughs, operational semantics, law-based testing, worked examples, migration guidance, and production adapter obligations. It distinguishes proven properties of the supplied implementation from assumptions required of future PostgreSQL, River, object-storage, and provider adapters.

2 Preface

2.1 Purpose

This book is both a design text and an implementation manual. It explains why the packages have their present shapes, how their kernels compose, what each extension point promises, and which laws a replacement implementation must preserve.

The intended reader is a Go engineer designing execution infrastructure for RAG indexing, evaluation, optimization, publication, or another artifact-oriented system. Familiarity with Go interfaces, contexts, hashing, transactions, and concurrent programming is assumed. Category theory is not assumed; the mathematical sections use only the amount needed to make engineering obligations precise.

2.2 Source basis

The implementation was developed from four supplied bodies of prior work:

1. the current `rag-ttc`, `ragkit`, `ragopt`, TTC, and GEC/CoinVault sources;
2. the *rag-ttc Semantic Architecture Handbook*;
3. the *rag-ttc Composition Pass Playbook*;
4. the *Compositional Durable Orchestration for Production Retrieval-Augmented Generation* thesis and its design transcript.

Those sources established several important boundaries:

- ordinary Go should remain the experiment and application composition language;
- semantic state, selective views, and operational traces are distinct;
- candidate accumulation should be lossless and mergeable where possible;
- durable plans must not be defined by a queue library;
- immutable artifacts and a small mutable publication boundary simplify recovery;
- attempts are at least once, while accepted transitions must be fenced;
- transport, cache, retry, evaluation, and publication contracts require separately versioned identities.

This handbook does not silently treat every proposal in the prior documents as implemented. The actual source tree is authoritative for implementation claims. Where the reference modules deliberately implement a smaller contract than a prior thesis, that difference is stated.

2.3 Validation status

Both standalone modules target Go 1.23 and use only the standard library. The supplied validation includes formatting, static analysis, ordinary tests, race-enabled tests, and executable demonstrations. The in-memory `ragjobs` Store is an executable reference semantics. The restartable file Store is a tested single-process durable interpreter. PostgreSQL migrations are supplied as a production contract, but no live PostgreSQL or River integration is claimed by this version.

The older source repositories target a later Go toolchain and external dependencies unavailable in the implementation environment. They were used as static compatibility and behavioral evidence rather than rebuilt as part of this work.

2.4 Reading routes

Readers interested primarily in architecture should read Chapters 1–4, 9–12, 17, and 18. Engineers implementing provider batching should focus on Chapters 5–8. Engineers implementing a durable Store should focus on Chapters 10–16 and 20. Reviewers interested in proofs and conformance should read Chapters 2–4, 8, 12, and 21.

3 1. The problem is two problems

3.1 1.1 Fine-grained execution

Consider an embedding stage over 30,000 chunk representations. The stage needs to:

- group requests into provider-sized batches;
- bound the number of concurrent calls;
- reuse previously captured successful values;
- coalesce duplicate semantic requests;
- retry only failed items;
- honor provider rate limits and a finite budget;
- restore outputs to the caller’s original order;
- emit useful operational observations.

All of this occurs inside one running attempt. The call stack and process lifetime are sufficient custody. If the process dies, a larger system decides whether the stage is run again.

This is the mandate of `flow`.

3.2 1.2 Durable orchestration

Now consider the complete indexing operation:

```
snapshot
  -> extract corpus
  -> chunk corpus
  -> (build lexical index || build dense index)
  -> assemble bundle
  -> verify bundle
  -> evaluate candidate
  -> apply quality gate
  -> publish alias
```

This computation may run for hours and cross deployments or machines. After a crash, the system must know:

- which source revision and configuration were being processed;
- which nodes committed immutable outputs;
- which worker currently has authority;
- whether a stale worker may still be running;
- which failed work is retryable;
- which results may be reused under the same semantic identity;
- whether budgets or deadlines permit more work;
- whether a candidate passed verification and policy;
- whether publication committed before an acknowledgement was lost.

This is not an in-process batching problem. It requires a persisted transition system.

This is the mandate of `ragjobs`.

3.3 1.3 The layer boundary

The principal composition rule is:

> A `ragjobs` handler may call `flow`. `flow` never owns the durable job graph, and `ragjobs` never needs to understand the fine-grained request loop.

A durable node should usually correspond to an independently meaningful immutable artifact or control decision worth recovering after process loss. A request inside `flow` should usually correspond to one semantically identified provider or algorithm invocation whose output can be reused independently.

The distinction is not based merely on size. It is based on **custody**.

- `flow` custody lasts for one process-level call to `Run`.
- `ragjobs` custody lasts across processes, attempts, restarts, and deployments.

3.4 1.4 Why not one universal workflow abstraction?

A universal abstraction tends to combine incompatible concerns:

- value transformation and process ownership;
- semantic identity and display names;
- per-item retry and node-level recovery;
- local worker count and distributed queue selection;
- batching and dependency causality;
- provider admission and publication serialization;
- pure composition and mutable lifecycle state.

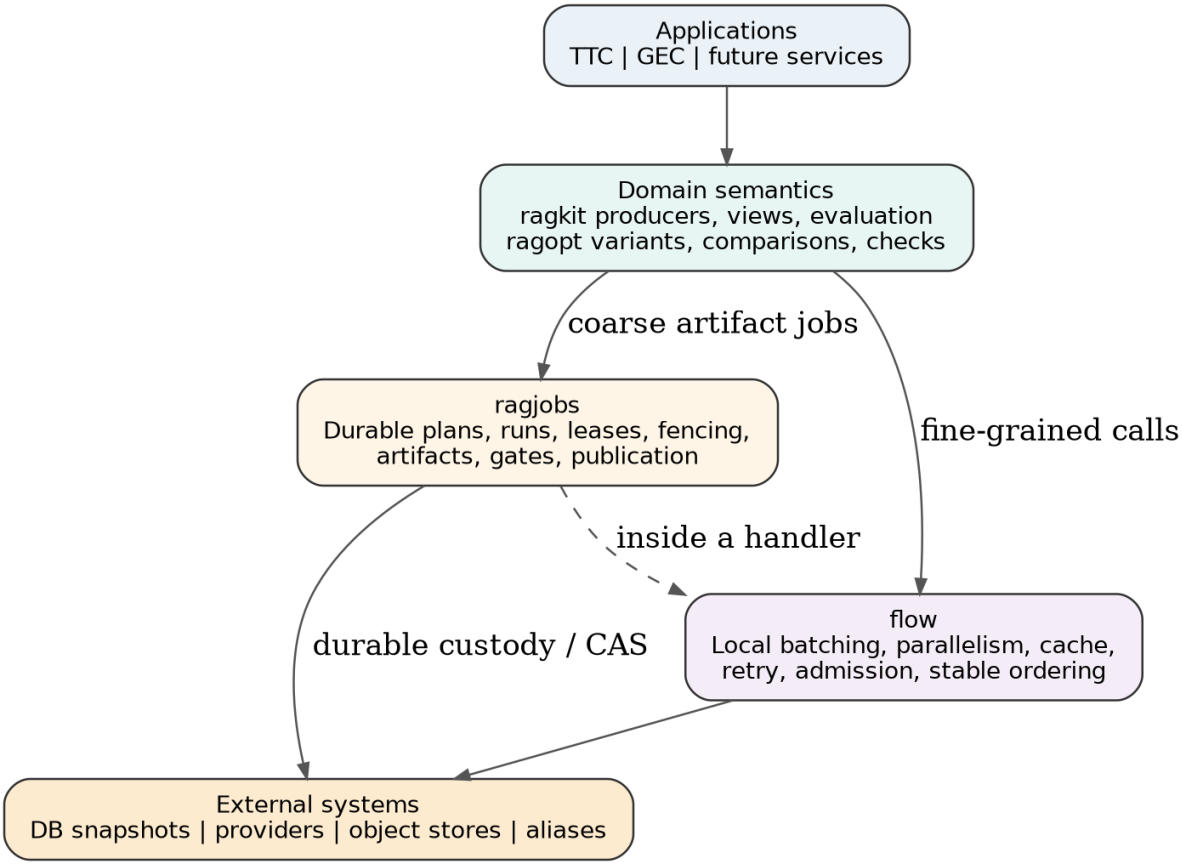

Figure 1: Execution layers. `ragjobs` owns durable coarse-grained orchestration; `flow` owns fine-grained work inside one handler attempt.

The result becomes difficult to explain and difficult to test. A change to an operational option can accidentally change semantic identity. A local callback can accidentally assign globally meaningful order. A convenient pipeline object can become an undeclared scheduler.

The two-module design removes that pressure. Each kernel has one observation level and one kind of lifecycle.

4 2. Semantic foundations

4.1 2.1 Information, views, and traces

Three data categories recur throughout both packages.

4.1.1 Semantic information

Semantic information is the value the computation means to establish. Examples include an embedding vector, an immutable chunk set, an index descriptor, a gate decision, or a published artifact reference.

Its identity excludes worker IDs, timestamps, retry counters, queue latency, and callback order.

4.1.2 View

A view is a deterministic projection of fixed semantic information under a versioned policy. Ranking, top-k selection, context packing, report rendering, and a quality gate are views.

Views may reorder or remove items. They are not generally monotone with respect to candidate-set growth.

4.1.3 Trace

A trace records execution history: cache hits, batch starts, attempts, workers, leases, heartbeats, completion order, delays, costs, and failures.

Two runs may have equal semantic output and unequal traces. This is expected, not a defect.

4.2 2.2 Three identities

The implementation separates three identity classes.

1. **Semantic identity** answers: *May one successful value substitute for another?*
2. **Plan identity** answers: *Which durable graph and policy was created?*
3. **Attempt identity** answers: *Which physical ownership interval performed this work?*

For example, changing `Workers: 8` to `Workers: 16` does not change an embedding request key. Changing the model revision does. Changing a durable node retry count changes the finalized plan, but does not necessarily change the semantic invocation key of the node. A recovered node gets a new attempt and fence, while retaining the same semantic key if its semantic input is unchanged.

Conflating these identities is a primary source of invalid cache reuse and irreproducible artifacts.

4.3 2.3 Content-addressed references

Both modules use domain-separated hashes. Abstractly, a semantic reference is:

$$\mathrm{Ref}(k, v, x) = \big(k, v, H(\mathrm{frame}(d, k, v, x))\big),$$

where:

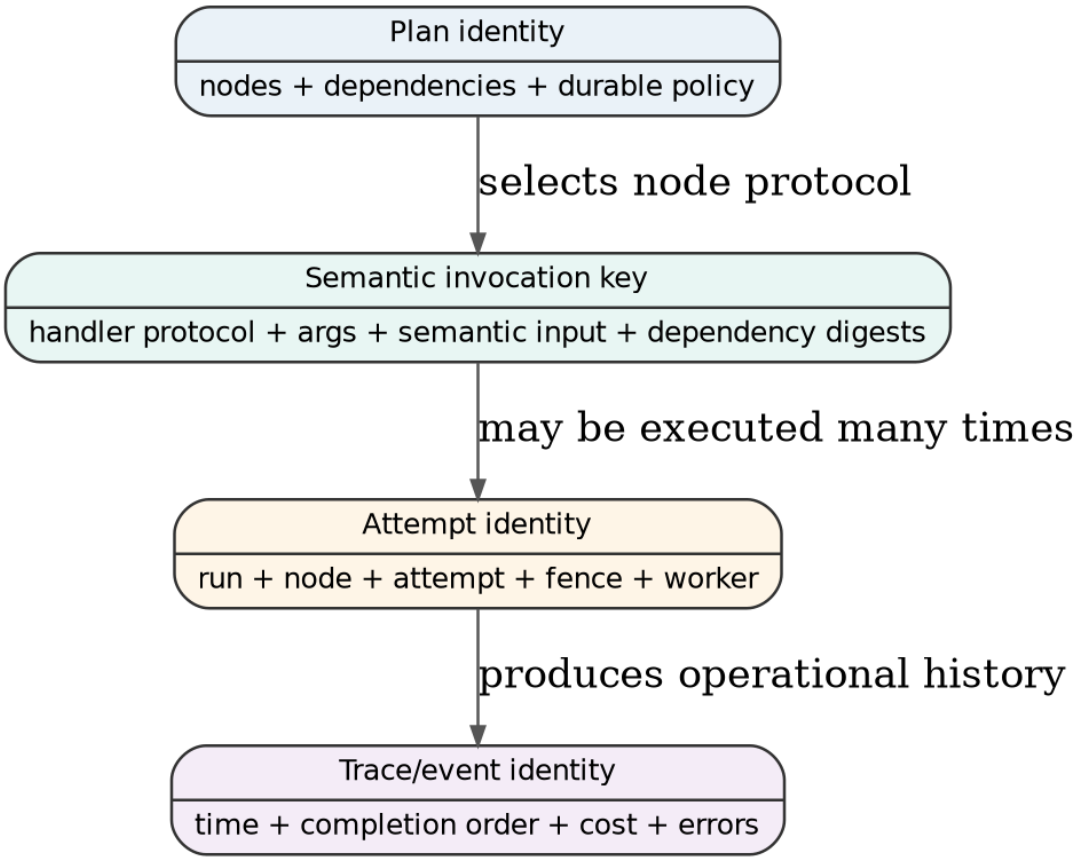

Figure 2: Three identity classes: semantic value or request, durable plan, and physical attempt/trace.

- k is a kind;
- v is a schema or protocol version;
- x is a canonical byte representation;
- d is a package-specific domain separator;
- `frame` length-prefixes fields, preventing concatenation ambiguity.

Domain separation prevents identical bytes in two conceptual domains from sharing an accidental identity. Versioning prevents an altered canonicalization or semantic contract from silently reusing old identities.

The hash is not a proof that two independently designed encoders mean the same thing. Correctness still depends on a documented equivalence relation for the canonical bytes.

4.4 2.4 Partial functions and semantic caches

A semantic cache is modeled as a partial function:

$$M : K \rightharpoonup V.$$

For one key k, either no successful value is known, or exactly one successful value is known. A second unequal value for the same key is a contradiction:

$$M(k) = v_1 \wedge v_1 \neq v_2 \quad \Rightarrow \quad \text{Put}(k, v_2) = \text{Conflict}\,.$$

This is stricter than an ordinary mutable cache. “Last writer wins” would hide incomplete identity, nondeterminism, corruption, or schema drift.

Cacheability is therefore a semantic assertion:

> For every world that agrees on the complete semantic input, all successful observations admitted under this key are equivalent under the promised output equality.

Operational traces and costs may differ between fresh execution and reuse.

4.5 2.5 Checked union and ACI laws

Several states in the system accumulate keyed information. The ideal merge is checked union:

```
new key                        -> insert
same key + same canonical value -> duplicate, accept
same key + different value      -> conflict
```

For compatible states, merge satisfies:

$$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c),$$

$$a \sqcup b = b \sqcup a,$$

$$a \sqcup a = a.$$

These are associativity, commutativity, and idempotence. They explain why duplicate result delivery, regrouped batches, and completion order can be semantically harmless when outputs are content addressed and conflicts fail closed.

The packages do not force every object into one generic semilattice type. They apply the law locally where it buys a concrete property: cache entries, immutable artifacts, normalized changes, dependency outputs, and candidate accumulation in adapters.

4.6 2.6 Pure functions remain ordinary functions

A pure deterministic value transformation belongs to ordinary Go:

```
normalized := Normalize(source)
chunks := Chunk(normalized)
manifest := Manifest(chunks)
```

No `Step` wrapper is needed merely to express composition. The framework appears only when one of the following is required:

- effect execution;
- batching or bounded concurrency;
- cache custody;
- resource admission;
- durable ownership;
- persisted dependency readiness;
- immutable artifact custody;
- atomic publication.

This principle keeps the semantic center visible.

5 3. A method for small extensible kernels

5.1 3.1 Begin with observation levels

Before introducing an interface, state what an observer is allowed to distinguish.

For `flow`, semantic equality ignores:

- worker count;
- batch boundaries;
- completion order;
- retry delay;
- cache hit versus fresh execution;
- observer callback order.

For `ragjobs`, node semantic equality ignores:

- run ID;
- worker ID;
- attempt number;
- fence;
- lease duration;
- event occurrence time;
- queue latency.

The trace retains these distinctions. The semantic result does not.

5.2 3.2 Put laws before fields

A field belongs in the kernel only when it participates in a stated invariant or transition. For example:

- `flow.Key` exists because cache substitution and duplicate coalescing require semantic identity;
- `ragjobs.Fence` exists because stale-worker exclusion requires a monotonically increasing ownership token;
- `Artifact.Ref` exists because semantic output identity must not depend on storage location;
- `ConcurrencyKey` exists because some incomparable DAG nodes still require serialization.

Conversely, a display label should not be overloaded as a unique identifier simply because it is convenient.

5.3 3.3 Prefer algebraic values to manager objects

A plan is data. A reference is data. A retry policy is data. A lease is data. A result is data.

The interpreter is the component that owns behavior. This makes it possible to:

- canonicalize and hash plans;
- persist historical protocols;
- test transitions independently of handlers;
- provide multiple Store implementations;
- replay or inspect values without starting workers.

5.4 3.4 Use narrow graft points

A graft point is an interface where the kernel truly needs an external interpretation. The reference modules use small interfaces such as:

```
type Processor[I, O any] interface {
    Process(context.Context, []I) ([]ItemResult[O], error)
}
```

```
type Store interface {
    CreateRun(context.Context, CreateRunRequest) (CreateRunResult, error)
    Claim(context.Context, ClaimRequest) ([]Lease, error)
    Heartbeat(context.Context, Lease, time.Time, time.Duration) (Lease, error)
    Complete(context.Context, Completion) error
    Fail(context.Context, Failure) error
    RecoverExpired(context.Context, time.Time, int) (int, error)
    CancelRun(context.Context, string, string, time.Time) error
    LookupCached(context.Context, string) (CachedResult, bool, error)
    GetRun(context.Context, string) (RunSnapshot, error)
    Events(context.Context, string, int64, int) ([]Event, error)
    ListRuns(context.Context, ListFilter) ([]RunSnapshot, error)
}
```

Each interface exists because alternate implementations are required. There is no global plugin lifecycle or hidden service locator. Applications assemble concrete implementations explicitly.

5.5 3.5 Keep optional algebra optional

`ragjobs.Plan` can be built directly as data. `Atom`, `Then`, and `Parallel` are optional syntax for reusable constructors and algebraic tests. They do not become the only way to express a plan.

This distinction matters. A small algebra can clarify laws without becoming a mandatory domain-specific language.

6 4. `flow`: mandate and non-goals

6.1 4.1 Contract

`flow` executes a finite slice of semantically identified typed requests inside one process-level call. It adds operational mechanics without claiming ownership of the larger application procedure.

Its core use case is:

> Execute many related requests with bounded concurrency, batching, content reuse, request-level retry, resource admission, stable output order, and bounded observation.

The package does not persist a DAG, schedule work across machines, recover leases, publish artifacts, or determine which application stages exist.

6.2 4.2 Why the old `Step` shape was rejected

A broad step object commonly begins as a typed function with a name. It then accumulates cache identity, workers, retry, batch policy, barriers, budgets, metering, callbacks, reporting, and composition metadata. Several problems follow.

First, semantic and operational settings become adjacent fields, encouraging invalid identity rules. Second, a local batching helper begins to look like a workflow engine. Third, composition laws become ambiguous because semantic outputs, reports, callbacks, and timing are all observable through the same object. Fourth, simple functions require framework wrappers even when no execution service is needed.

The new `flow` removes the central `Step[I,O]` concept. The actual operation is a `Processor[I,O]`; the actual request identity is a `Key`; the interpreter is `Run`; resource and custody behaviors are grafts on the interpreter.

6.3 4.3 Kernel inventory

The user can understand the kernel by reading four files:

Concept	Role
`Key`	caller-owned semantic request identity
`Call[I]`	semantic key plus local typed input
`Processor[I,O]`	aligned batch effect
`Run`	operational interpreter

Everything else is either a policy value, an extension interface, a concrete extension, or a convenience type.

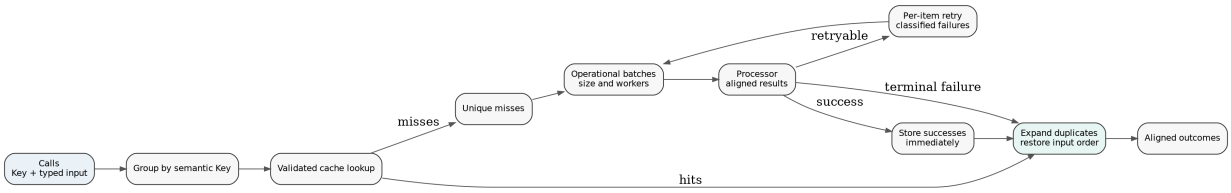

Figure 3: `flow` separates semantic request identity from operational batching, retry, cache, admission, and trace.

7 5. `flow` semantic API

7.1 5.1 `Key`

The key type is intentionally transparent:

```
type Key struct {
    Kind    string `json:"kind"`
    Version string `json:"version"`
    Digest  string `json:"digest"`
}
```

Construction uses a domain-separated SHA-256 digest over a kind, version, and canonical payload. The caller, not the executor, owns the equivalence relation.

```go
key, err := flow.JSONKey(
    "embedding/request",
    "v2",
    struct {
        Model          string `json:"model"`
        ModelRevision  string `json:"model_revision"`
        Representation string `json:"representation"`
        TextDigest     string `json:"text_digest"`
        Dimensions     int    `json:"dimensions"`
        Normalize      bool   `json:"normalize"`
    }{
        Model:          "embed-x",
        ModelRevision:  "2026-07-17",
        Representation: "raw",
        TextDigest:     textDigest,
        Dimensions:     1536,
        Normalize:      true,
    },
)
```

A correct key contains every setting that may alter the successful value. Typical semantic fields include:

- model and pinned model revision;
- prompt and response schema digests;
- input text or artifact identity;
- output dimension;
- normalization;
- tool mode;
- decoding and reasoning settings;
- index snapshot and query policy;
- adapter semantic version.

Typical operational exclusions include:

- worker count;
- batch size;
- retry count and delay;
- rate limiter implementation;
- trace observer;
- output path;
- wall-clock occurrence time.

The safest key contract is:

> Two calls share a `Key` exactly when `flow` is permitted to execute one successful result and substitute it for both.

7.2 5.2 Canonicalization and versioning

`JSONKey` uses `encoding/json`. Go's encoder sorts map keys, but persistent protocols still require discipline around floats, omitted fields, custom marshaling, and schema evolution.

When equivalence changes, change the key version. Do not silently reinterpret persisted keys.

For highly stable protocols, define a specific canonical request struct rather than hashing a large runtime configuration object. This makes field classification reviewable and prevents secrets or operational settings from entering persisted identity accidentally.

7.3 5.3 `Call[I]`

```
type Call[I any] struct {
    Key   Key
    Input I
}
```

`Input` is the local typed value needed to invoke the processor. `Key` is the stable semantic identity. The two need not be the same representation.

For example, `Input` may contain a provider client-ready byte slice or an in-memory object, while the key contains only canonical semantic fields. This avoids forcing generic serialization into the hot path.

Calls with equal keys are coalesced inside one `Run`. They must also report equal admission units. If the caller constructs one key for semantically unequal inputs, the package cannot repair that error. It will execute one and return the same outcome to all duplicates, exactly as the key contract permits.

7.4 5.4 `Processor[I,O]`

```
type Processor[I, O any] interface {
    Process(context.Context, []I) ([]ItemResult[O], error)
}
```

A processor consumes one physical batch and returns aligned per-item results.

```
type ItemResult[T any] struct {
    Value T
    Err   error
}
```

There are two failure levels:

- a top-level processor error means the whole physical batch failed;
- an `ItemResult.Err` means one item failed while others may have succeeded.

This distinction is essential for partial retry. A provider batch endpoint can return vectors for 126 items and errors for 2. `flow` commits and caches the 126 successes immediately, then retries only the 2 failures.

The alignment law is strict:

$$|\text{Process}(xs)| = |xs|$$

whenever the top-level error is nil. Violating the law is an executor custody error, not an item failure.

`ProcessorFunc` adapts a batch function. `Each` adapts an ordinary per-item function without adding composition semantics:

```
processor := flow.Each(func(ctx context.Context, input Input) (Output, error) {
    return compute(ctx, input)
})
```

7.5 5.5 `Outcome` and error levels

`Run` returns aligned outcomes:

```
type Outcome[T any] struct {
    Key      Key
    Value    T
    Err      error
    Cache    CacheStatus
    Attempts int
}
```

Item failures are values in `Outcome.Err`. The top-level error from `Run` is reserved for custody or interpreter failure, such as:

- invalid keys;
- corrupt cache entries;
- semantic cache conflicts;
- failed cache persistence;
- malformed processor alignment;
- context cancellation affecting the interpreter;
- invalid configuration.

This separation prevents one ordinary failed request from erasing successful sibling results.

8 6. `flow.Run` operational semantics

8.1 6.1 Signature

```
func Run[I, O any](
    ctx context.Context,
    executor Executor,
    calls []Call[I],
    processor Processor[I, O],
    config Config[I, O],
) ([]Outcome[O], Report, error)
```

`Executor` contains run-scoped collaborators:

```
type Executor struct {
    Cache      Cache
    Admissions []Admitter
    Observer   Observer
    Clock      Clock
}
```

`Config` contains operational policy and output encoding:

```
type Config[I, O any] struct {
    Workers   int
    BatchSize int
    Retry     RetryPolicy
    Codec     Codec[O]
    Units     func(I) int
}
```

No field in `Executor` or `Config` is automatically included in request identity. Semantic settings must already be represented by each call's `Key`.

8.2 6.2 Execution phases

The interpreter implements the following phases.

8.2.1 Phase 1: validate and group

Every key is validated. Calls are grouped by key in first-occurrence order. The group stores all original positions so the final result can be expanded back to caller order.

If n calls contain u unique keys, the executor performs at most u semantic executions in the run.

8.2.2 Phase 2: cache lookup

For each unique key, the executor queries the optional cache. A hit must pass:

- stored-key equality;
- encoded-value digest validation;
- output decoding.

A corrupt entry is not treated as a miss. Failing closed avoids silently replacing a damaged reproducibility record with an unrelated fresh result.

8.2.3 Phase 3: form a retry wave

All pending unique items enter an attempt wave. Items are ordered by stable unique position and partitioned into batches of `BatchSize`.

Batch boundaries are operational. They do not alter the semantic key or caller order.

8.2.4 Phase 4: execute batches concurrently

Up to `Workers` batches execute concurrently. Before one physical batch starts, every configured `Admitter` receives a `Work` value containing keys, attempt number, item count, and caller-defined units.

The processor receives only typed inputs. It has no access to output positions or cache internals unless the application deliberately includes such data in `I`.

8.2.5 Phase 5: commit successes immediately

Each successful item is marked complete. When a cache exists, its encoded value is stored immediately under the semantic key. This matters when a later sibling or later retry fails: successful work is not lost merely because the whole call did not become globally successful.

8.2.6 Phase 6: classify failures

Every item failure is classified independently. Unknown errors fail closed as permanent. Only explicitly transient or rate-limited errors retry by default.

If retry is permitted and attempts remain, the item enters the next wave. Successful siblings do not.

8.2.7 Phase 7: wait once for the next wave

The next delay is at least the policy backoff and any provider-supplied `RetryAfter`. The run waits before executing the pending set again.

8.2.8 Phase 8: expand outcomes

Each unique item outcome is copied to every original call position sharing its key. Caller order is restored exactly.

8.3 6.3 Stable order under concurrency

Let input calls be $c_0, \dots, c_{n-1}$. `Run` promises output outcomes $o_0, \dots, o_{n-1}$ such that:

$$o_i.\,\text{Key} = c_i.\,\text{Key}\,.$$

This is independent of physical completion order. The package does not promise deterministic observer event order under multiple workers; that order is part of the operational trace.

8.4 6.4 Duplicate transparency

For a set of call positions D_k sharing key k, the executor performs one unique request and expands its outcome:

$$\forall i, j \in D_k, \quad (o_i.\,\text{Value}, o_i.\,\text{Err}) = (o_j.\,\text{Value}, o_j.\,\text{Err}).$$

This property is stronger than merely suppressing duplicate cache writes. It prevents duplicate in-flight work inside one run.

8.5 6.5 Partial retry

Suppose a batch contains item keys $\{a, b, c\}$, and only b fails transiently. The next wave contains $\{b\}$ rather than the entire batch. In set notation:

$$P_{r+1} = \{k \in P_r \mid \text{failed}(k) \wedge \text{retryable}(k) \wedge \text{attempts}(k) < m\}.$$

This is both a cost property and a correctness property. Successful observations are immutable facts for the current run and should not be repeated merely because another aligned item failed.

8.6 6.6 Cache transparency

For a correct cacheable processor, replacing a fresh successful execution with a validated cache hit preserves the semantic outcome:

$$\text{Semantic}(\text{Run}_{hit}(k)) = \text{Semantic}(\text{Run}_{fresh}(k)).$$

The report and trace differ. A cache hit has no current provider attempt, and the operational cost may be lower.

8.7 6.7 Report semantics

```
type Report struct {
    Calls      int
    Unique     int
    CacheHits  int
    Fresh      int
    Failed     int
    Attempts   int
    Batches    int
    Duplicates int
}
```

A `Report` is operational summary data. It is not a source of semantic output identity. Two executions with equal output values may have different reports because of cache state, retries, or batch policy.

9 7. `flow` graft points

9.1 7.1 Cache

```
type Cache interface {
    Get(context.Context, Key) (CacheEntry, bool, error)
    Put(context.Context, CacheEntry) error
}
```

The contract is deliberately stronger than a generic key/value store. `Put` must reject an unequal value already associated with the same semantic key.

Included implementations are:

- `cache.Memory`, suitable for tests and process-local reuse;
- `cache.File`, a content-validated local persistent cache.

The file cache uses create-if-absent publication semantics. Concurrent equal writes are harmless; an unequal existing value produces a semantic conflict.

A future Redis, SQLite, PostgreSQL, or object-store cache must preserve the same partial-function law.

9.2 7.2 Codec

A cache stores bytes, while `Processor` returns typed values. `Codec[O]` is the serialization graft. `JSONCodec[O]` is supplied.

Encoding belongs outside the semantic key unless the encoded byte format itself is the promised output equality. The key should identify the meaning of the request; the cache entry separately validates the exact captured bytes.

9.3 7.3 Retry classifier

```
type Classifier interface {
    Classify(error) Decision
}
```

The default classifier retries only errors explicitly wrapped as transient or rate limited. This prevents accidental retry storms for malformed input, unsupported schemas, authorization failures, and code defects.

The package provides:

```
flow.Permanent(err)
flow.Transient(err)
flow.RateLimited(delay, err)
```

Provider adapters should translate their native errors at the boundary. Core code should not parse strings to infer retry semantics.

9.4 7.4 Admission

```
type Admitter interface {
    Admit(context.Context, Work) error
}
```

Admission happens for every physical batch attempt, including retries. This allows accurate enforcement of:

- token or record budgets;
- provider request limits;
- rate limits;
- test fault injection;
- resource-class policies.

Included adapters include a finite `Budget`, a `TokenBucket`, and a `Chain`.

Admission is operational. An admission failure can stop or fail execution, but the admission policy does not enter semantic request keys unless it changes the successful value rather than merely whether or when the request runs.

9.5 7.5 Observer

An `Observer` receives bounded events such as cache hits, batch starts, item success, item failure, and scheduled retry.

Observers cannot veto or mutate execution. This restriction prevents an observability callback from becoming a hidden semantic stage. A callback may append keyed events, update metrics, or create spans. It should not assign semantic rank, select winners, or mutate source artifacts.

9.6 7.6 Clock

The clock controls retry waits and event timestamps. A fake clock makes retry tests deterministic without making wall-clock time part of request identity.

9.7 7.7 Explicit assembly

The application constructs extensions directly:

```
executor := flow.Executor{
    Cache: cache.NewFile(cacheDir),
    Admissions: []flow.Admitter{
        admission.NewBudget(2_000_000),
        admission.NewTokenBucket(20, 40),
    },
    Observer: telemetry,
}
```

There is no dynamic global plugin registry. The composition is visible in ordinary Go.

10 8. `flow` laws and tests

10.1 8.1 Operational invariance

Under a deterministic processor and fixed cache snapshot, changing workers or batch size must not change aligned semantic outcomes:

$$\mathrm{Values}(\mathrm{Run}_{w_1,b_1}(C)) = \mathrm{Values}(\mathrm{Run}_{w_2,b_2}(C)).$$

The reports and event traces may differ.

This law requires that the processor's successful value be independent of batch composition and concurrency. If a provider changes semantics based on batch order or neighboring inputs, that behavior must be represented in the semantic request contract or declared unsupported.

10.2 8.2 Permutation qualification

`flow` preserves caller order, so arbitrary input permutations naturally produce correspondingly permuted outputs. The stronger set-level property is:

$$\text{MapByKey}(\text{Run}(\pi C)) = \text{MapByKey}(\text{Run}(C))$$

for permutations π when input order is declared non-semantic.

10.3 8.3 Retry transparency

For a request that eventually returns one fixed successful value, inserting retryable failures before success does not alter the admitted value:

$$\text{value}(f, f, \ldots, v) = v.$$

Attempt counts, waits, cost, and trace differ.

10.4 8.4 Budget accounting

Admission is called on every physical attempt. If `Units(i)` is u_i, then a conservative budget observes:

$$\text{spent} = \sum_{\text{physical attempts } a} \sum_{i \in a} u_i.$$

A retry is not free merely because it has the same semantic key.

10.5 8.5 Cache functionality

For every reachable cache state:

$$\forall k, \quad |\{v \mid (k, v) \in M\}| \leq 1.$$

Tests deliberately attempt unequal writes and corrupted reads. Both must fail rather than degrade into misses.

10.6 8.6 Alignment custody

A processor returning the wrong number of aligned results violates the protocol. The interpreter returns a top-level error and does not guess how to associate results.

10.7 8.7 Race tests

The race suite runs concurrent batch workers and cache access. Race freedom is necessary but not sufficient: the semantic invariance tests separately verify schedule independence under the declared processor assumptions.

A mutex proves serialized access, not deterministic meaning. Both kinds of tests are required.

10.8 8.8 Example: embedding 30,000 representations

A realistic setup is:

```
type Request struct {
    Text       string
    TextDigest string
}

type Vector struct {
    Values []float32 `json:"values"`
}

calls := make([]flow.Call[Request], 0, len(representations))
for _, representation := range representations {
    key, err := flow.JSONKey("embedding", "v3", struct {
        ModelRevision string `json:"model_revision"`
        TextDigest    string `json:"text_digest"`
        Dimensions    int    `json:"dimensions"`
    }{
        ModelRevision: modelRevision,
        TextDigest:    representation.Digest,
        Dimensions:    1536,
    })
    if err != nil {
        return err
    }
    calls = append(calls, flow.Call[Request]{
        Key: key,
        Input: Request{
            Text:       representation.Text,
            TextDigest: representation.Digest,
        },
    })
}

outcomes, report, err := flow.Run(
    ctx,
    executor,
    calls,
    providerBatch,
    flow.Config[Request, Vector]{
        Workers:   8,
        BatchSize: 128,
        Retry: flow.RetryPolicy{
            MaxAttempts: 4,
            Backoff: flow.Backoff{
                Base:       time.Second,
                Cap:        30 * time.Second,
                Multiplier: 2,
            },
        },
        Codec: flow.JSONCodec[Vector]{},
        Units: func(request Request) int {
            return estimateTokens(request.Text)
        },
    },
)
```

The application then validates all outcomes and assembles one vector-manifest artifact. That arti-

fact, not each individual embedding request, is normally the durable `ragjobs` node output.

10.9 8.9 Migration from a broad step API

A practical migration sequence is:

1. extract the actual typed function or provider batch call from the old `Step`;
2. define a complete semantic request-key constructor in the domain package;
3. map old per-item inputs to `Call[I]`;
4. map batching logic to `Processor[I,O]`;
5. move retry classification into the provider adapter;
6. move cache storage behind `Cache`;
7. move budgets and rate limits behind `Admitter`;
8. move callbacks to a non-mutating `Observer`;
9. leave pure stage composition in ordinary Go;
10. move durable dependencies and recovery to `ragjobs` rather than rebuilding them in `flow`.

The compatibility wrapper should be temporary and tested against the old behavior. The target API should not preserve accidental coupling merely to avoid adapting callers.

11 9. `ragjobs`: mandate and architecture

11.1 9.1 Contract

`ragjobs` is a transport-independent durable orchestration kernel for finite, coarse, artifact-producing computations.

It owns:

- immutable canonical plan definitions;
- dependency readiness;
- durable runs and node states;
- attempts, leases, and fences;
- retry and lease recovery;
- cancellation and terminal monotonicity;
- run budgets and concurrency keys;
- semantic result reuse;
- immutable output lineage;
- event history;
- trigger deduplication;
- the contracts required for evaluation, quality gates, and publication.

It does not define RAG algorithms, provider APIs, product routing, evaluation metrics, or queue transport behavior.

11.2 9.2 Durable job granularity

A computation deserves a durable node when its successful output is independently meaningful and worth recovering after process loss.

Typical durable nodes are:

Operation	Durable boundary?	Reason
snapshot source	yes	establishes consistency boundary and immutable source identity
normalize one document	usually no	fine-grained local work
extract complete corpus	yes	reusable immutable artifact

Operation	Durable boundary?	Reason
chunk one document	usually no	inner computation
produce complete chunk set	yes	common branch input
embed one chunk	usually no	handled by `flow` cache and retry
build dense index/manifest	yes	expensive recoverable artifact
build lexical index	yes	independent branch and artifact
assemble bundle	yes	explicit compatibility check and immutable candidate
verify bundle	yes	named evidence worth retaining
evaluate one case	usually no	shard internally unless cases are very expensive
evaluate a shard	yes	retry/cost unit and immutable evidence
aggregate evaluation	yes	deterministic fan-in result
apply quality gate	yes	explicit reproducible decision
publish alias	yes	small mutable critical section

Too-fine durable nodes overload the control database and expose provider noise as global orchestration state. Too-coarse nodes lose useful recovery, cache, cost, and audit boundaries.

11.3 9.3 Reference indexing plan

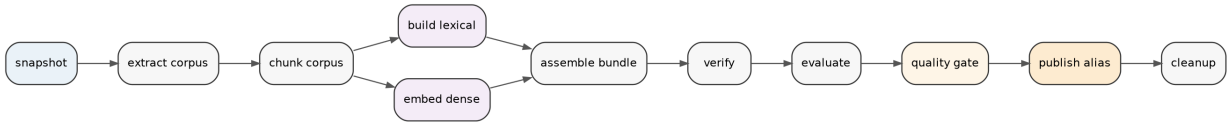

Figure 4: Reference durable indexing DAG. Lexical and dense branches are independent after the shared chunk-set artifact.

The graph is illustrative rather than hard-coded. `ragjobs` has no package named `indexer` in its kernel. Application plan constructors define node kinds, versions, arguments, ports, retry, cacheability, cost bounds, queues, and failure modes.

11.4 9.4 Mutable control plane, immutable data plane

The architecture separates:

11.4.1 Mutable control plane

- run and node state;
- attempts and leases;
- event sequence;
- semantic-cache index;
- trigger deduplication;
- concurrency-key ownership;
- publication aliases.

11.4.2 Immutable data plane

- source snapshots;
- corpora;
- chunk sets;
- lexical indexes;
- vector manifests and indexes;
- assembled bundles;
- verification evidence;
- evaluation shards and reports;
- gate decisions.

The control plane stores references to data-plane artifacts rather than mutating large active values. Publication changes one small alias after verification.

12 10. References, artifacts, and semantic outputs

12.1 10.1 `Ref`

```go
type Ref struct {
    Kind    string `json:"kind"`
    Version string `json:"version"`
    Digest  string `json:"digest"`
}
```

`NewRef` hashes in-memory canonical bytes. `NewRefReader` hashes a stream while enforcing an exact declared size, giving the same identity without loading a large artifact into memory.

The equality promise is semantic and schema-qualified. A corpus manifest and an evaluation report with identical bytes still have different references because their kinds differ.

12.2 10.2 `Artifact`

```go
type Artifact struct {
    Ref       Ref               `json:"ref"`
    MediaType string            `json:"media_type,omitempty"`
    Size      int64             `json:"size,omitempty"`
    Location  string            `json:"location,omitempty"`
    Metadata  map[string]string `json:"metadata,omitempty"`
}
```

`Ref` is semantic identity. Location and metadata describe custody and provenance.

This distinction permits the same immutable value to move between a local filesystem and object storage without changing downstream semantic invocation keys. Conversely, changing artifact bytes changes the reference even when the path remains the same.

12.3 10.3 Named output maps

A node returns named artifacts. `OutputDigest` canonicalizes the map by output name and artifact reference while ignoring storage location and trace metadata.

For outputs $O = \{(n_i, r_i)\}$:

$$\text{OutputDigest}(O) = H\big(\text{sort}_n(n_i, r_i)\big).$$

This digest is used for cache conflict detection and dependency identity.

12.4 10.4 Artifact immutability law

Once an artifact with reference r is admitted, opening its location must produce bytes whose canonical identity is r. If a storage object at that location changes, verification fails. The artifact is corrupt; it is not a new version of the same artifact.

Mutable names are handled by publication aliases, not by overwriting content-addressed objects.

13 11. Plans as canonical durable values

13.1 11.1 `NodeSpec`

A durable node specification is data:

```
type NodeSpec struct {
    ID             string
    Handler        HandlerRef
    Args           json.RawMessage
    DependsOn      []string
    Inputs         []Port
    Outputs        []Port
    Queue          string
    Priority       int
    Timeout        time.Duration
    Retry          RetryPolicy
    Cache          CacheMode
    Failure        FailureMode
    MaxCost        int64
    ConcurrencyKey string
    Labels         map[string]string
}
```

The fields fall into several classes.

13.1.1 Semantic protocol

- `Handler.Kind` and `Handler.Version`;
- canonical `Args`;
- named artifact input/output contracts.

13.1.2 Causality

- `ID`;
- `DependsOn`.

13.1.3 Durable operational policy

- queue and priority;
- timeout;
- retry schedule;
- cache assertion;
- failure propagation mode;
- maximum cost;
- concurrency key.

These operational policies are part of plan identity because they explain how this historical run was intended to execute. They are not automatically part of the node semantic result key.

13.2 11.2 Handler protocol versioning

`HandlerRef{Kind, Version}` names a durable protocol. Persisted plans may outlive a deployment. Therefore:

- semantic changes require a new version;
- old handlers must remain available while retained nonterminal plans require them;
- missing handlers are permanent protocol failures;
- a bug fix may retain a version only when it restores the already documented semantics rather than changing them.

The handler version is closer to a message protocol version than a package release number.

13.3 11.3 Ports

Ports are nominal artifact boundaries:

```
type Port struct {
    Name string
    Type string
}
```

Finalization checks that every declared input type is supplied by a direct dependency. The reference version intentionally performs nominal type validation rather than maintaining a schema registry.

Handlers and artifact readers remain responsible for structural payload validation. A future registry can refine the contract without changing the plan graph model.

13.4 11.4 `Finalize`

`Finalize` is a pure normalization and identity operation:

1. clone caller-owned values;
2. set the plan schema version;
3. clear any supplied plan ID;
4. trim and validate stable identifiers;
5. canonicalize node arguments;
6. apply defaults;
7. sort dependency sets, ports, and nodes;
8. reject unknown dependencies, duplicate IDs, invalid ports, and cycles;
9. hash canonical plan data;
10. store the resulting plan ID.

The purity property is important: callers can reuse or compare their input value without discovering that finalization mutated slices or maps.

13.5 11.5 Canonical plan identity

If two plan values differ only in orderings declared irrelevant or in omitted fields replaced by identical defaults, finalization produces the same ID.

Let $N(P)$ be plan normalization and H the reference hash. Then:

$$\text{PlanID}(P) = H(N(P)).$$

For every representation permutation π that preserves the plan's declared meaning:

$$N(\pi P) = N(P) \quad \Rightarrow \quad \text{PlanID}(\pi P) = \text{PlanID}(P).$$

Changing retry, queue, timeout, cache mode, node protocol, arguments, dependencies, or ports changes the plan value and therefore generally changes its identity.

13.6 11.6 Direct plan data versus composition helpers

Applications may construct a `Plan` directly. Optional fragments provide:

```
snapshot := ragjobs.Atom(snapshotSpec)
extract  := ragjobs.Atom(extractSpec)
lexical  := ragjobs.Atom(lexicalSpec)
dense    := ragjobs.Atom(denseSpec)

prefix, _   := ragjobs.Then(snapshot, extract)
branches, _ := ragjobs.Parallel(lexical, dense)
flow, _     := ragjobs.Then(prefix, branches)
plan, _     := flow.Plan("knowledge-index", "v1", labels)
```

`Then` adds every left exit as a direct dependency of every right entry. `Parallel` is disjoint union.

13.7 11.7 Composition laws

For fragments with pairwise disjoint node IDs:

13.7.1 Sequential associativity

$$(P; Q); R = P; (Q; R).$$

Both sides contain the same nodes, internal edges, and boundary edges.

13.7.2 Identity

$$I; P = P = P; I,$$

where I is the empty fragment.

13.7.3 Parallel associativity and symmetry

$$(P \parallel Q) \parallel R = P \parallel (Q \parallel R),$$

$$P \parallel Q = Q \parallel P,$$

modulo canonical ordering.

`Parallel` states absence of a causal edge. It does **not** prove arbitrary side effects commute. Concurrency keys, artifact isolation, provider idempotency, and handler contracts establish the operational preconditions for safe parallel execution.

14 12. Semantic invocation identity

14.1 12.1 Run input

A run input contains:

```
type RunInput struct {
    Trigger Trigger
    Values  map[string]json.RawMessage
    Labels  map[string]string
    Budget  Budget
}
```

The trigger separates semantic source position from operational occurrence data:

```
type Trigger struct {
    Kind       string
    Source     string
    Cursor     string
    Payload    json.RawMessage
    DedupKey   string
    OccurredAt time.Time
}
```

The semantic run-input reference includes:

- trigger kind;
- source;
- cursor;
- canonical payload;
- canonical run values.

It excludes:

- trigger deduplication key;
- occurrence time;
- labels;
- budget.

The excluded fields may alter whether or when a run executes, but not the meaning of a successful node result.

14.2 12.2 Node semantic key

For node specification n, semantic run input i, and direct dependency output map D, the key is:

$$K(n, i, D) = H\big(\text{handler}(n), \text{args}(n), \text{semanticInput}(i), \text{sort}(D)\big).$$

In code, `SemanticKey` binds:

- handler kind and version;
- canonical node args;
- semantic run-input reference;
- each direct dependency's named output digest, sorted by dependency node ID.

It excludes attempt, fence, worker, lease expiration, trace, and current budget state.

14.3 12.3 Why direct dependency digests matter

A source cursor may identify a lower bound rather than an immutable snapshot. For example, a mutable database snapshot requested after revision 42 may observe revision 45 by the time a consistent read begins.

The snapshot node should be non-cacheable unless it can reproduce exactly the declared source state. It emits an immutable snapshot artifact containing the actual observed revision. Downstream

semantic keys bind the snapshot output digest, so they identify the actual source value rather than merely the trigger request.

14.4 12.4 Plan identity versus semantic key

A plan records intended operational policy. A node key records substitution equivalence of a successful result.

Changing `MaxAttempts` from 3 to 5 changes plan identity. It need not change the semantic result key. Changing the handler version, canonical args, source input, or dependency artifact changes both the relevant semantics and the node key.

This is intentional. Historical orchestration remains explainable without destroying valid content reuse.

15 13. Operational state machine

15.1 13.1 States

Runs have states:

```
queued -> running -> succeeded
                  -> failed
                  -> canceled
```

Nodes have states:

```
blocked -> available -> running -> succeeded
                               \-> available   (retry)
                               \-> failed
blocked ----------------------> skipped
blocked/available/running ------> canceled
```

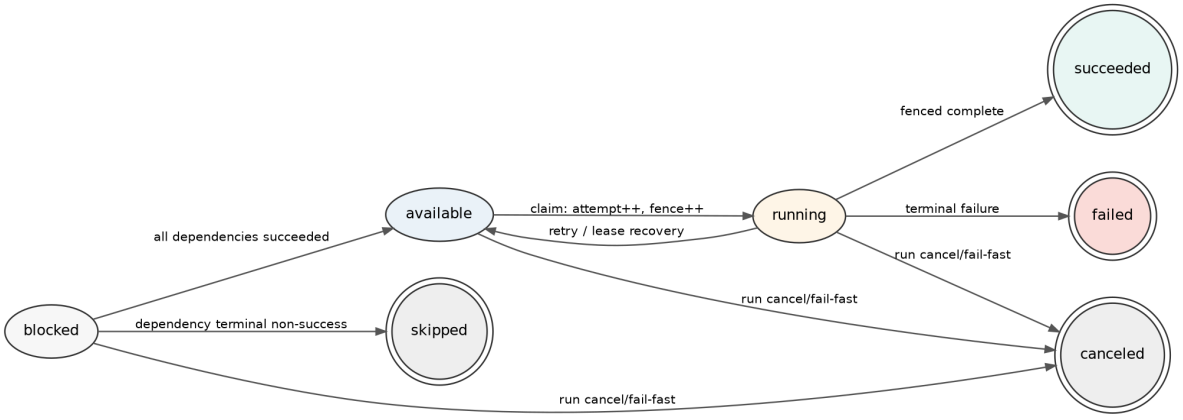

Figure 5: Durable node and run state transitions. Ownership transitions are fenced, and terminal states are monotone.

Terminal run states are `succeeded`, `failed`, and `canceled`. Terminal node states are `succeeded`, `failed`, `skipped`, and `canceled`.

15.2 13.2 The Store is the semantics

```go
type Store interface {
    CreateRun(context.Context, CreateRunRequest) (CreateRunResult, error)
    Claim(context.Context, ClaimRequest) ([]Lease, error)
    Heartbeat(context.Context, Lease, time.Time, time.Duration) (Lease, error)
    Complete(context.Context, Completion) error
    Fail(context.Context, Failure) error
    RecoverExpired(context.Context, time.Time, int) (int, error)
    CancelRun(context.Context, string, string, time.Time) error
    LookupCached(context.Context, string) (CachedResult, bool, error)
    GetRun(context.Context, string) (RunSnapshot, error)
    Events(context.Context, string, int64, int) ([]Event, error)
    ListRuns(context.Context, ListFilter) ([]RunSnapshot, error)
}
```

Every mutating method is one atomic transition in a conforming durable implementation. The SQL schema alone is not the semantics; transaction code and conditional predicates must realize the same preconditions and postconditions as the reference Store.

15.3 13.3 Configuration model

Abstractly, the Store state is:

$$C = (R, N, A, E, M, D, L, B, K, t),$$

where:

- R maps run IDs to run records;
- N maps run/node pairs to node records;
- A is attempt history;
- E is the append-only per-run event sequence;
- M is the semantic cache;
- D is trigger deduplication state;
- L is lease state inside nodes;
- B is consumed and reserved budget state;
- K is active concurrency-key ownership;
- t is authoritative time.

A transition is a partial function $C \to C'$ whose preconditions are checked atomically.

15.4 13.4 CREATE

`CreateRun`:

1. finalizes and validates the plan;
2. normalizes the immutable run input;
3. applies trigger deduplication scoped by plan and source;
4. creates one run record;
5. creates every node record;
6. marks roots available and non-roots blocked;
7. appends `run_created` and root `node_ready` events.

A duplicate trigger returns the existing run ID without creating a second run.

The deduplication key is operational run-creation identity. It is not the semantic identity of node results.

15.5 13.5 Readiness

A blocked node becomes available exactly when all direct dependencies succeeded. If any dependency reaches a terminal non-success state, the blocked node becomes skipped.

Repeated propagation computes a least fixed point because skipping one node may make its descendants permanently impossible.

Let F be one propagation pass over node states. Starting from S_0:

$$S_{n+1} = F(S_n).$$

The finite DAG and monotone terminal transitions ensure the process stabilizes:

$$\exists m, \quad S_{m+1} = S_m.$$

15.6 13.6 CLAIM

A node is claimable only when:

- it is available;
- `AvailableAt` is not in the future;
- its run is nonterminal;
- its queue is accepted by the worker;
- the run deadline remains open;
- attempt budget remains;
- maximum cost can be reserved;
- its concurrency key is not active.

An accepted claim atomically:

1. moves a queued run to running;
2. increments node attempt;
3. increments node fence;
4. sets owner and lease expiration;
5. computes the semantic invocation key;
6. reserves the node's declared maximum cost;
7. increments run attempt usage;
8. acquires the concurrency key;
9. appends attempt history and events;
10. returns a lease with immutable direct-dependency outputs.

15.7 13.7 Lease as capability

```go
type Lease struct {
    RunID        string
    PlanID       string
    NodeID       string
    Attempt      int
    Fence        uint64
    Worker       string
    ExpiresAt    time.Time
    Spec         NodeSpec
    Input        RunInput
    Dependencies map[string]map[string]Artifact
    SemanticKey  string
}
```

A lease is a time-bounded capability. Every completion, failure, or heartbeat must present the current run, node, attempt, fence, worker, and unexpired authority.

The fence increases strictly on each claim:

$$f_{n+1} > f_n.$$

Once a later claim exists, an earlier worker cannot satisfy the current-lease predicate.

15.8 13.8 HEARTBEAT

A heartbeat succeeds only for the current live lease. It extends expiration and appends an event.

A worker that loses heartbeat authority may continue executing because cancellation is cooperative. Its later Store completion is rejected. External mutable effects therefore need their own idempotency or fencing protocol; Store fencing protects Store state, not an unrelated provider.

15.9 13.9 COMPLETE

Completion requires:

- current matching live lease;
- nonnegative actual cost no greater than declared maximum;
- outputs matching declared port names and artifact types;
- valid artifact references;
- a valid named output digest;
- no unequal cached output already stored for the semantic key.

The transition:

1. finishes the attempt;
2. releases reserved cost;
3. adds actual cost;
4. records outputs, output digest, semantic key, and reuse status;
5. inserts a cache entry when declared cacheable;
6. clears owner and lease;
7. releases concurrency-key ownership;
8. appends success or reuse event;
9. propagates ready and skipped descendants;
10. finalizes the run if all nodes are terminal.

A repeated completion after a successful commit fails the lease predicate. The state transition occurs once even if the acknowledgement is lost.

15.10 13.10 FAIL and RETRY

A failed attempt releases its reservation, records measured cost and classification, clears ownership, and either:

- returns the node to available at a durable retry time; or
- makes it terminally failed.

Retry requires:

- a retryable failure class;
- remaining node attempts;
- remaining run attempt budget;
- remaining deadline;
- remaining cost budget.

`FailRun` cancels remaining nonterminal work after terminal node failure. `ContinueRun` preserves independent branches and skips causal descendants; the run ultimately remains failed if any required node failed or skipped.

15.11 13.11 RECOVER

`RecoverExpired` finds running nodes whose lease expired and processes them through the same failure/retry decision using the `lease_expired` class.

Recovery does not merely clear an owner field. It records attempt outcome, releases reservation and concurrency ownership, applies retry policy, appends events, and preserves fencing history.

15.12 13.12 CANCEL

Run cancellation atomically:

- marks all nonterminal nodes canceled;
- finishes live attempts;
- releases reservations and concurrency keys;
- clears leases;
- appends cancellation events;
- marks the run canceled.

Old leases are immediately invalid because both run and node state preconditions fail.

15.13 13.13 FINALIZE

A run succeeds exactly when every node succeeded. It fails when all nodes are terminal and at least one required node failed, skipped, or was canceled due to failure. Explicit operator cancellation produces the distinct canceled state.

16 14. Safety properties

16.1 14.1 Dependency safety

Property. A node can be claimed only after every direct dependency succeeded.

Argument. Non-roots begin blocked. The only transition from blocked to available is the all-dependencies-succeeded branch of propagation. `Claim` requires available state.

This property is stronger than queue ordering. Even if a transport delivers a child early or twice, the Store refuses to grant authority until semantic readiness holds.

16.2 14.2 Stale-worker exclusion

Property. After ownership transfers to a later claim, no completion or failure from an earlier claim is accepted.

Argument. A later claim increments attempt and fence. Completion and failure require exact equality with current worker, attempt, fence, state, and live expiration.

This is a control-plane safety property. It cannot reverse an external side effect already committed by an old worker.

16.3 14.3 Terminal monotonicity

Property. Once a run or node reaches a terminal state, no legal transition returns it to a nonterminal state.

Administrative repair creates a new run or future derived-run protocol. It does not reopen historical terminal state in place.

16.4 14.4 Single accepted outcome per attempt

For a tuple $(run, node, attempt, fence)$, at most one completion or failure transition is accepted. The first transition changes the running state and clears the lease; later submissions fail the current-lease predicate.

16.5 14.5 Event-prefix integrity

Events for one run form a gap-free positive sequence:

$$1, 2, \ldots, n.$$

The reference Store appends under its transaction lock. A PostgreSQL interpreter must increment the run-local counter and insert the event in the same transaction as the state transition.

16.6 14.6 Cache functionality

One semantic key maps to at most one output digest. An unequal second result is `ErrResultConflict`, a semantic defect rather than a transient failure.

Likely causes include:

- incomplete semantic inputs;
- moving model aliases;
- nondeterministic handler behavior marked cacheable;
- inconsistent canonicalization;
- mutable dependency state not captured by artifact digests;
- corruption.

16.7 14.7 Conservative budget safety

Let C_c be consumed cost and C_r reserved maximum cost. Before a claim with node bound g:

$$C_c + C_r + g \leq C_{max}.$$

Completion replaces reservation g with actual $c \leq g$. Under honest conservative declarations, the Store never starts work whose worst-case successful completion would exceed the positive run maximum.

The Store can reject a reported cost greater than the declared maximum after execution. It cannot recover money already spent by a defective handler, so bounds must be conservative.

16.8 14.8 Concurrency-key exclusion

At most one running node owns a nonempty concurrency key. Typical keys include:

```
publish:gec:knowledge:production
compact:tenant-42:index-A
provider-account:embedding-primary
```

A concurrency key is operational serialization. It does not imply semantic equality between nodes sharing the key.

16.9 14.9 Eventual terminality assumptions

Safety alone does not guarantee progress. Eventual terminality requires:

- a finite plan;
- compatible workers eventually polling every required queue;
- finite handler attempts or lease expiration;
- fair recovery;
- finite retry and budget bounds;
- eventual release or expiry of concurrency keys;
- an available durable Store.

Under these assumptions, every run eventually succeeds, fails, or is canceled.

17 15. Worker and handler protocol

17.1 15.1 Handler

```
type Handler interface {
    Run(context.Context, Invocation) (Result, error)
}
```

An invocation contains immutable semantic inputs plus the current operational capability:

```
type Invocation struct {
    RunID        string
    PlanID       string
    NodeID       string
    Spec         NodeSpec
    Input        RunInput
    Dependencies map[string]map[string]Artifact
    SemanticKey  string
    Attempt      int
    Fence        uint64
    Worker       string
}
```

The handler returns semantic outputs and measured cost. It does not update node state directly.

17.2 15.2 Registry

The application-owned `Registry` maps `(kind,version)` to a handler. Duplicate registration is rejected.

This is a graft table, not a dynamic plugin runtime. Applications register implementations explicitly at startup, and worker deployment reports which protocols it supports.

17.3 15.3 Worker interpreter

The transport-neutral Worker:

1. claims leases from accepted queues;
2. checks semantic cache for cacheable nodes;
3. resolves the versioned handler;
4. creates an attempt timeout;
5. heartbeats while the handler runs;
6. converts panics to classified failures;
7. persists completion or failure;

8. leaves parent-shutdown ambiguity to lease recovery.

A handler failure that is successfully persisted is not a worker infrastructure error. The worker can continue processing other jobs.

17.4 15.4 Cache reuse

Before running a cacheable handler, the Worker calls `LookupCached`. A hit is submitted through ordinary fenced completion with `Reused: true` and zero current cost.

The Store remains authoritative. A worker cannot bypass current lease checks merely because it found a cache value.

17.5 15.5 Parent shutdown

If the worker process is shutting down, an in-flight handler may not have reached a safely classified domain result. The Worker deliberately leaves the attempt unresolved rather than persisting cancellation as a semantic failure. Lease recovery later decides retry or terminal outcome.

This preserves at-least-once semantics and avoids confusing process shutdown with application rejection.

17.6 15.6 Panics

Panics are recovered and classified as `panic`, permanent by default. Retrying arbitrary code defects immediately is usually harmful. A narrowly understood adapter may map a known condition differently, but the kernel does not retry panics automatically.

18 16. Store implementations

18.1 16.1 Memory Store

`store/memory` serializes each method under one mutex. It is the executable reference semantics for:

- plan and input normalization;
- run creation and deduplication;
- readiness;
- claims and fences;
- heartbeats;
- completion and cache conflicts;
- failure and retry;
- lease recovery;
- cancellation;
- budgets;
- concurrency keys;
- event sequences;
- finalization.

The implementation is intentionally direct. Production adapters should match its observable transitions rather than inventing slightly different lifecycle behavior.

18.2 16.2 Exported reference state

The Memory Store can marshal and restore its complete state. Restore validates:

- state schema;
- run identity;
- finalized plan identity;

- normalized input;
- event-prefix integrity;
- node membership;
- running lease completeness;
- concurrency-key uniqueness;
- cache output digests;
- dedup references.

This mechanism supports the restartable file Store and differential state-machine testing.

18.3 16.3 File Store

`store/file` wraps the Memory Store and atomically persists the complete state after each successful mutating transition. If persistence fails, it restores the previous in-memory state so the public call does not report a transition that was not durably recorded.

The file Store is useful for:

- local development;
- command-line tools;
- demonstrations;
- single-process deployments with modest state;
- conformance tests.

It is not a multi-process queue. File-level persistence does not provide distributed claim contention or database-grade availability.

18.4 16.4 Store conformance

`storetest` supplies reusable behavioral checks. Any PostgreSQL or alternative implementation should run the same cases.

High-value conformance scenarios include:

- only roots are initially claimable;
- a child becomes ready only after all dependencies succeed;
- a recovered stale lease cannot complete;
- equal semantic work is reused across runs;
- unequal results for one semantic key conflict;
- cost reservation prevents over-admission;
- concurrency keys exclude simultaneous owners;
- independent branches continue under `ContinueRun`;
- trigger dedup returns the existing run;
- event sequences remain gap free;
- restart preserves active leases and later recovery.

18.5 16.5 PostgreSQL refinement obligation

A PostgreSQL Store should treat each mutating method as one transaction. Typical mechanisms include:

- `SELECT ... FOR UPDATE` for run/node transitions;
- `FOR UPDATE SKIP LOCKED` for claim candidate selection;
- unique constraints for semantic keys and trigger deduplication;
- conditional updates containing current state, owner, attempt, and fence;
- a partial unique index or advisory lock for concurrency keys;
- database time for lease authority;
- transactionally appended events.

`READ COMMITTED` can be sufficient when every mutable row is explicitly locked and all accepted transitions have fenced predicates. Serializable isolation is not a substitute for protocol checks.

18.6 16.6 River refinement obligation

River or another queue may transport physical dispatches. The semantic source of truth remains the `ragjobs` Store.

Exactly one layer must own physical retry timing. Two valid profiles are:

- direct Store profile: `ragjobs` owns availability times, claims, retries, and recovery;
- River profile: River owns physical dispatch retry timing, while `ragjobs` validates semantic readiness, leases, outputs, cache, lineage, and terminal state.

Do not independently schedule the same failed attempt in both systems.

19 17. Immutable artifacts and publication

19.1 17.1 Content-addressed artifact store

`artifactfs` writes immutable objects to a path derived from the semantic reference. The logical protocol is:

1. stream bytes into a temporary object while hashing;
2. construct or verify the declared reference;
3. synchronize the file;
4. publish into the content-addressed namespace;
5. if the object already exists, verify exact content equality;
6. return an `Artifact` descriptor.

The implementation rejects path traversal and verifies objects on open.

A production object-store adapter should preserve the same semantics using immutable keys, multipart checksums, and conditional creation where available.

19.2 17.2 Why artifacts, not directories in progress

A mutable shared build directory creates ambiguous crash states:

- which files were complete;
- whether lexical and dense outputs describe the same chunk set;
- whether a reader observed half-written state;
- whether a retry may overwrite valid work;
- whether an old worker can still mutate the directory.

An immutable artifact has one identity and one validation rule. Assembly occurs in scratch space and only the finished descriptor is admitted to the Store.

19.3 17.3 Publication alias

Publishing an index is intentionally a separate, small mutable operation. An alias record contains a name, active artifact, generation, optional source revision, and provenance.

A publication request includes:

- alias name;
- candidate artifact;
- expected current generation or digest;
- source revision when ordered;
- run/node/fence provenance;

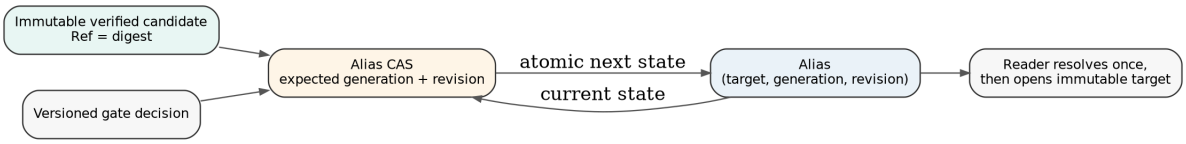

Figure 6: Immutable artifacts are built and verified before a small compare-and-swap alias transition exposes the candidate.

- verification or gate evidence references.

19.4 17.4 Compare-and-swap semantics

For current alias state (d_0, g_0, r_0) and proposed (d, g, r):

1. if $(d, r) = (d_0, r_0)$, return idempotent success;
2. if ordered revisions are enabled and $r \leq r_0$ with a different candidate, reject stale publication;
3. if expected digest or generation does not match, reject conflict;
4. otherwise write $(d, g_0 + 1, r)$ atomically.

The transition is linearizable when performed under one atomic lock or database row update.

19.5 17.5 Publication is not cacheable

Artifact construction may be semantically reusable. Publication is a mutable control transition and is not replaced by a semantic cache hit.

An ambiguous publication outcome is reconciled by reading the alias:

- already points to same digest and revision: idempotent success;
- points to newer revision: stale old run;
- points to unrelated generation: conflict requiring policy;
- no committed change: retry when safe.

19.6 17.6 Rollback

Rollback does not mutate or rebuild an old artifact. It performs an audited alias transition to a prior verified digest and creates a new alias generation.

When normal automatic publication enforces monotone source revisions, operational rollback is a privileged override protocol. It must record actor, incident, reason, previous generation, and target artifact.

19.7 17.7 Garbage collection

Content-addressed artifacts require reachability-based retention. Roots include:

- active aliases;
- rollback retention sets;
- nonterminal runs;
- retained run outputs;
- semantic-cache entries;
- evaluation evidence;
- manually pinned or quarantined objects.

Deletion should use mark, delay, recheck, then sweep. A failed candidate may have greater diagnostic value than an old successful intermediate.

20 18. Change capture and triggers

20.1 18.1 The dual-write problem

A source database mutation and a scheduler submission are two effects. If they occur in different transactions, one can commit without the other.

The initial production pattern is a transactional outbox:

Figure 7: Source-side transactional outbox, at-least-once relay, contiguous adoption, immutable build, and monotone publication.

The source transaction:

1. updates domain rows;
2. allocates or records a stream revision;
3. inserts an index-relevant outbox event;
4. commits all three together.

A relay copies events at least once to the control plane. Deduplication makes replay harmless.

20.2 18.2 Source revision

A source stream is identified by a stable scope such as:

```
(source, system, corpus)
```

and has a monotonically increasing positive revision. The revision is allocated in the same transaction as the domain change and outbox rows.

Timestamps remain useful for latency. They are not a safe total order for concurrent source transactions.

20.3 18.3 Normalized changes

The `trigger` package normalizes source changes and constructs deterministic batch identities. Changes may carry:

- stream identity;
- revision;
- event ID;
- entity kind and key;
- operation;
- non-sensitive metadata.

Normalization sorts stable sets and removes duplicate event IDs. Relay order does not determine batch identity.

20.4 18.4 Contiguous adoption

Suppose the inbox contains revisions 10 and 12, while 11 is delayed. Adopting 10–12 as one interval would conceal the gap.

The production coalescer begins at the committed cursor plus one and stops at the first missing revision. Later events remain pending.

If the committed cursor is r, the next emitted batch covers:

$$[r+1,\ r+k]$$

such that every revision in the interval exists, and either $r+k$ is the current high watermark or $r+k+1$ is the first gap.

The committed cursor advances only after run adoption is durably recorded.

20.5 18.5 Snapshot consistency

The snapshot handler is normally the only indexing node permitted to read mutable source truth. It starts a consistent read, verifies that the source revision is at least the requested contiguous revision, reads all relevant data, and emits a content-addressed snapshot artifact recording the actual observed revision.

Downstream nodes consume only immutable artifacts.

A mutable current-state snapshot is non-cacheable by default because rerunning the same lower-bound cursor later may observe a different state. A repository commit or database time-travel snapshot can be cacheable when every relevant dependency is pinned.

20.6 18.6 Configuration changes

Index invalidation is not limited to source rows. Changes to chunking, analyzers, representation policy, embedding model, evaluation set, prompt, or quality policy require explicit versioned run values or handler versions.

A configuration-only run may reuse source snapshots and deterministic upstream artifacts when semantic keys prove equivalence.

21 19. Evaluation, quality gates, and optimization

21.1 19.1 Evaluation as artifacts

Evaluation consumes immutable identities:

- candidate bundle;
- optional baseline bundle;
- suite revision;
- route and policy configuration;
- metric implementations;
- optional judge configuration.

It emits immutable evidence. This makes evaluation usable by publication gates, experiments, and incident review without rerunning providers.

21.2 19.2 Deterministic fan-out/fan-in

A durable evaluation plan can be:

```
enumerate cases
   -> shard-0000
   -> shard-0001
   -> ...
   -> shard-N
   -> aggregate
   -> compare
   -> gate
```

Case assignment should be deterministic, for example:

$$\text{shard}(q) = H(\text{caseID}(q)) \bmod N.$$

The partition manifest records suite identity, shard count, and assignments. Aggregation verifies complete expected shard coverage and rejects duplicate or conflicting case evidence.

21.3 19.3 Stochastic judges

An LLM judge is not assumed to be a pure mathematical function. Default cacheability should be off unless the organization adopts an explicit recorded-observation equivalence.

A judged artifact should record:

- exact provider/model identity where available;
- inference fingerprint;
- rubric and prompt digests;
- candidate/case input references;
- raw response and parsed decision;
- retries, latency, cost, and adapter version.

The first accepted observation can be immutable evidence without claiming every fresh execution would agree.

21.4 19.4 Quality gates

A quality gate is a deterministic view over named evidence and a versioned policy. It should return a decision artifact with reasons rather than silently turning quality rejection into infrastructure failure.

Useful checks include:

- structural validity;
- complete expected case coverage;
- minimum metric floors;
- no-regression tolerances;
- critical-case zero-failure rules;
- slice-specific floors;
- latency and cost ceilings;
- manual approval requirements.

The gate policy must be identified by immutable digest or protocol version. It must not look up mutable policy by an unversioned name during execution.

21.5 19.5 `ragopt` as a workload, not a scheduler

Optimization defines variants, coordinates, evaluations, comparisons, and checks. `ragjobs` supplies durable execution.

A coordinate can be modeled as:

$$\text{Coord} = (case, variant, repeat).$$

Cells are immutable results keyed by coordinates. A complete run is checked union of cells. Resume computes expected coordinates minus completed coordinates.

Candidate evaluations can fan out in parallel, aggregate, compare against baseline, and apply promotion checks. This reuses durable ownership and artifacts without making `ragopt` responsible for leases or queue semantics.

22 20. Composing `flow` inside `ragjobs`

22.1 20.1 The pattern

A coarse durable handler can use `flow` for its fine-grained internal work:

```
registry.Register(
    ragjobs.HandlerRef{Kind: "rag.embed-dense", Version: "v2"},
    ragjobs.HandlerFunc(func(
        ctx context.Context,
        invocation ragjobs.Invocation,
    ) (ragjobs.Result, error) {
        chunkSet, err := loadChunkSet(invocation.Dependencies)
        if err != nil {
            return ragjobs.Result{}, ragjobs.Permanent(err)
        }

        calls, err := embeddingCalls(chunkSet, invocation.Spec.Args)
        if err != nil {
            return ragjobs.Result{}, ragjobs.Permanent(err)
        }

        outcomes, report, err := flow.Run(
            ctx,
            localExecutor,
            calls,
            providerBatch,
            localConfig,
        )
        if err != nil {
            return ragjobs.Result{}, ragjobs.Transient(err)
        }

        vectors, err := flow.Values(outcomes)
        if err != nil {
            class, delay := classifyEmbeddingFailure(err)
            return ragjobs.Result{}, ragjobs.Classified(class, delay, err)
        }

        artifact, err := writeVectorManifest(vectors, report)
        if err != nil {
            return ragjobs.Result{}, ragjobs.Transient(err)
        }
        return ragjobs.Result{
            Outputs: map[string]ragjobs.Artifact{"vectors": artifact},
            Cost:    measuredProviderCost(outcomes),
        }, nil
    }),
)
```

The local `flow` cache can reuse individual embeddings. The durable `ragjobs` cache can reuse the completed vector-manifest artifact. The two cache layers have different keys and custody scopes.

22.2 20.2 Nested retry discipline

Nested retry is valid only when scopes differ and multiplication is bounded.

- `flow` retries short-lived request-level provider failures;

- `ragjobs` retries the whole node after process death, exhausted inner retry, database outage, or classified node failure.

The worst-case attempt product must be included in cost and latency planning.

Exactly one durable transport owns the next node-attempt time. An inner provider retry sleeps only within the live handler attempt.

22.3 20.3 Failure mapping

A handler maps local outcomes to one durable result:

- all required fine-grained work succeeded: emit immutable aggregate artifact;
- a known retryable provider condition remains: return classified transient or rate-limited node failure;
- malformed immutable input or unsupported schema: permanent failure;
- partial diagnostic mode: emit an explicit incomplete artifact only when the handler protocol declares that representation valid.

Do not convert unavailable or failed fine-grained work into an indistinguishable empty artifact.

22.4 20.4 Budget mapping

`flow` admission counts actual physical request attempts. The durable node declares a conservative `MaxCost`, and the handler returns measured total cost.

This creates two safeguards:

- the local layer prevents runaway provider calls inside one attempt;
- the durable Store prevents starting attempts whose declared worst case exceeds the run budget.

22.5 20.5 Trace linkage

`flow` events should carry semantic request keys and be linked to the current `ragjobs` run/node/attempt/fence in telemetry context. The durable event log need not contain every fine-grained event. It may store a summary artifact or trace reference to keep control-plane state bounded.

23 21. Extensibility without a framework monoculture

23.1 21.1 Graft-point map

The principal extension interfaces are:

Layer	Graft point	Responsibility
`flow`	`Processor[I,O]`	provider or algorithm batch effect
`flow`	`Cache`	semantic request-result partial function
`flow`	`Admitter`	local rate, budget, or resource policy
`flow`	`Classifier`	provider/domain retry mapping
`flow`	`Observer`	bounded operational telemetry
`ragjobs`	`Handler`	versioned coarse application protocol
`ragjobs`	`Store`	atomic durable state transitions
`ragjobs`	artifact store	immutable bytes and verification
`ragjobs`	publication store	alias CAS and revision policy
`ragjobs`	trigger relay/coalescer	source-change adoption
deployment	queue adapter	physical wakeup and delivery

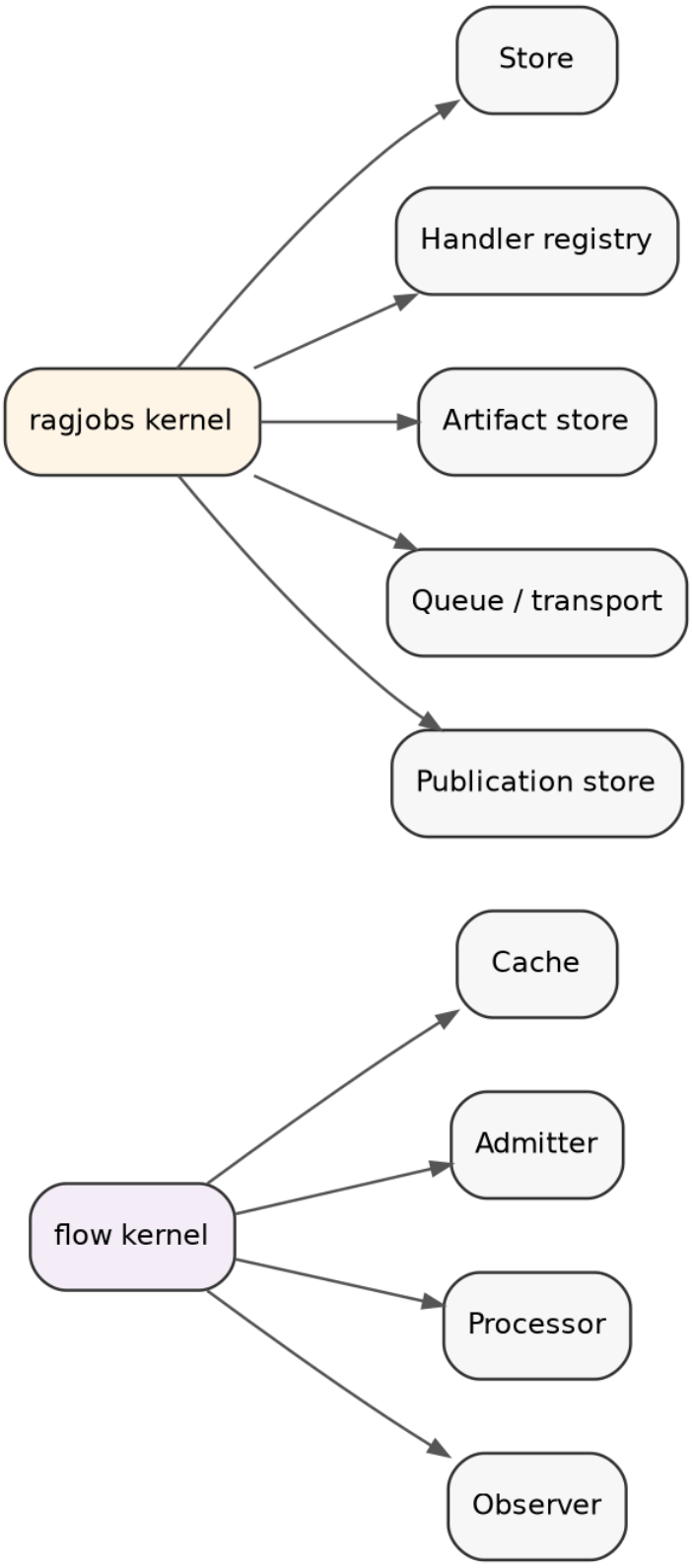

Figure 8: The kernels expose narrow graft points. Applications assemble concrete implementations explicitly.

23.2 21.2 Plugin means package plus interface

A plugin should normally be a Go package implementing one narrow interface. It should not imply:

- dynamic shared-object loading;
- global discovery;
- implicit lifecycle hooks;
- unordered registration;
- hidden dependency injection;
- a universal configuration schema.

Explicit assembly is preferable:

```
registry := ragjobs.NewRegistry()
must(registry.Register(snapshotRef, source.NewSnapshotHandler(...)))
must(registry.Register(embedRef, dense.NewHandler(...)))
must(registry.Register(publishRef, publisher.NewHandler(...)))

worker := ragjobs.Worker{
    Store:    durableStore,
    Registry: registry,
    ID:       workerID,
    Queues:   []string{"cpu", "embedding"},
}
```

The graph remains inspectable. The application owns which capabilities exist.

23.3 21.3 Capability extension versus kernel expansion

Add a kernel field only when every interpreter must understand it to preserve semantics. Otherwise prefer an extension artifact, handler argument, label, or policy package.

Examples:

- adding `Fence` belongs in the kernel because every Store must reject stale completions;
- adding GPU memory estimates might begin as labels or a scheduler extension until admission semantics are stable;
- adding a new quality check belongs in the evaluation/gate package, not the Store;
- adding W3C PROV serialization belongs in an artifact adapter, not the run state machine.

23.4 21.4 Small interfaces and capability discovery

Avoid one enormous backend interface where implementations support unrelated optional features. Use a core interface plus optional capability assertions when necessary.

For example, a future transport might support transactional dispatch insertion or queue pause. These capabilities need not contaminate `Store` if they are not required for semantic correctness.

23.5 21.5 Version every persisted protocol

Persisted values outlive code. Version:

- reference domains;
- plan schema;
- handler protocols;
- canonical args;
- artifact schemas;
- event payload schemas when interpreted externally;
- publication aliases;
- cache equivalence.

A version change is explicit incompatibility or migration, not an implementation detail.

24 22. Designing elegant semantically sound APIs

24.1 22.1 State the equivalence first

Before writing a cache key, answer:

> Under what exact conditions may one captured successful result substitute for another request?

Before writing a plan ID, answer:

> Which changes make this a different historical execution contract?

Before writing an artifact reference, answer:

> Which bytes and schema define immutable equality?

Before writing a report equality test, answer:

> Is this semantic output, a view, or an operational trace?

The field list follows from the answers.

24.2 22.2 Separate values from computations

A value is immutable information. A computation may fail, retry, consume resources, emit traces, or alter durable control state.

Do not put attempt counters or timestamps into semantic values merely because the code has them available. Do not hide an external effect behind a function presented as pure.

In practical Go:

- `Ref`, `Artifact`, `Plan`, `NodeSpec`, `Lease`, and `Result` are values;
- `Processor`, `Handler`, `Store`, and publication CAS interpret effects.

24.3 22.3 Separate denotation from interpretation

The plan says what depends on what and which protocol should execute. The Store and Worker say how the plan progresses operationally. River or PostgreSQL says how workers are awakened and transitions are persisted.

Changing transport should not change plan meaning.

24.4 22.4 Prefer data plus interpreter

A data-oriented protocol yields:

- inspectability;
- stable hashing;
- persistence;
- replay;
- multiple interpreters;
- independent test generation;
- easier migration.

A manager object with hidden mutable fields is harder to compare and resume.

24.5 22.5 Reject impossible ambiguity early

Examples of fail-closed behavior include:

- same semantic key with unequal output;
- same reference with unequal bytes;
- processor output length different from input length;
- plan cycle;
- unknown direct dependency;
- missing declared artifact port;
- stale fence;
- old source revision publication;
- running restored node without owner or lease expiration.

Silently guessing makes later failures harder to classify.

24.6 22.6 Do not overclaim algebra

Sequential function composition is associative at the value level. Re-grouped operational pipelines may have different traces, retries, and resource behavior.

Parallel graph composition identifies potential independence. It does not make shared mutable side effects commute.

An append-only event set is monotone, while a current-state projection over tombstones or last-write rules may not be.

A citation reference may be valid without proving entailment.

Precise qualification is part of API quality.

24.7 22.7 Make partial failure explicit

Distinguish:

- success;
- missing/unavailable;
- retryable failure;
- permanent failure;
- canceled;
- skipped by causal failure;
- budget exhaustion;
- semantic conflict;
- quality rejection.

A quality gate rejection is usually a valid decision artifact, not infrastructure failure. A failed source read is not an empty corpus.

24.8 22.8 Keep the happy path ordinary

Small semantic kernels should not force advanced terminology on routine application code. A developer should be able to:

- build calls;
- call `flow.Run`;
- define node specs;
- finalize a plan;
- register handlers;
- start a worker;
- inspect a run.

The mathematical model exists to constrain behavior and tests, not to make every call site ceremonial.

25 23. Worked durable indexing example

25.1 23.1 Plan

The included `ragjobs/examples/indexing` demonstration creates eleven nodes:

1. `snapshot`;
2. `extract-corpus`;
3. `chunk-corpus`;
4. `build-lexical`;
5. `embed-dense`;
6. `assemble-bundle`;
7. `verify-bundle`;
8. `evaluate-candidate`;
9. `quality-gate`;
10. `publish`;
11. `cleanup`.

Lexical and dense branches share the chunk-set dependency and may run concurrently. Assembly requires both.

25.2 23.2 Handler behavior

The demo handlers produce typed immutable artifacts in a local content-addressed store. The dense handler intentionally returns a rate-limited failure on its first attempt. The node is durably retried, receives a larger fence, and succeeds on its second attempt.

Every other node succeeds once.

25.3 23.3 Publication

The gate decision carries the verified candidate. The publication handler performs a revision-aware alias compare-and-swap. The active alias advances only after the immutable candidate was assembled, opened, verified, evaluated, and accepted.

25.4 23.4 Observed result

The validated demonstration produces:

- one successful run;
- eleven succeeded nodes;
- twelve attempts;
- one intentional rate-limited retry;
- thirty-eight durable events;
- a published alias at generation 1 and source revision 42.

The exact timestamps, temporary directory, and worker trace are operational. The plan identity and artifact references are stable for the same canonical inputs.

26 24. Migration for TTC, GEC, ragkit, and ragopt

26.1 24.1 Migration principle

Treat the existing code as behavioral evidence and compatibility input, not as the target API. Replace one boundary at a time while retaining known-good artifacts and semantic comparisons.

Compare separately:

- candidate or artifact semantics;
- selected views;
- answer/evaluation contracts;
- operational traces;
- cost and latency.

Do not declare compatibility solely because final answer strings match.

26.2 24.2 Phase 0: identity hardening

Before moving orchestration, fix known incomplete identities:

- canonical evidence digest consistency;
- complete generation inference fingerprints;
- fusion constants and algorithm versions;
- stable stage IDs distinct from display names;
- deterministic evidence selection and labels where required.

Migration onto a new runtime does not repair an incomplete semantic key automatically.

26.3 24.3 Phase 1: introduce shared references

Adapt current documents, chunks, indexes, bundles, suites, policies, and generated observations to explicit semantic references.

Do not immediately replace domain types. Add constructors and verification at artifact boundaries.

26.4 24.4 Phase 2: migrate local execution

For each existing `flow.Step` path:

- extract the processor;
- define semantic request keys;
- move local execution options into `flow.Config` and grafts;
- compare outputs under multiple worker and batch settings;
- preserve current caches through a compatibility adapter or deliberate version reset.

Start with embedding and generated representation paths, where the local executor provides the largest operational value.

26.5 24.5 Phase 3: extract coarse services

Refactor command bodies into versioned handler services:

- snapshot source;
- extract corpus;
- chunk/represent;
- build lexical;
- build dense;
- assemble/open/verify bundle;
- run evaluation;

- gate;
- publish.

A first migration may use one coarse `build-bundle` node when clean internal artifact APIs do not yet exist. Decompose later at genuine recovery boundaries rather than fabricating stages.

26.6 24.6 Phase 4: shadow durable runs

Run `ragjobs` plans for real triggers without publishing. Compare:

- source revision and snapshot manifests;
- chunk IDs and content digests;
- lexical/vector coverage;
- bundle manifests;
- retrieval outputs;
- evaluation evidence;
- costs and durations.

Investigate every semantic cache conflict. Keep candidate artifacts for forensic comparison.

26.7 24.7 Phase 5: manual publication

Introduce alias publication separately from current mutable bundle paths. Require explicit verification and manual promotion. Practice conflict handling, idempotent replay, quarantine, and rollback.

26.8 24.8 Phase 6: change capture

For GEC/CoinVault, write MySQL outbox events in the same transaction as index-relevant domain changes. Relay to the control plane at least once and adopt contiguous revisions.

For TTC repository corpora, resolve mutable branch names to immutable commits before run creation. Application database changes use an outbox.

26.9 24.9 Phase 7: automatic gated promotion

Enable automatic publication only after:

- Store conformance and crash tests;
- source no-loss/gap tests;
- versioned handler and artifact protocols;
- stable quality policy;
- alias CAS and rollback drills;
- serving-generation observability;
- acceptable shadow correctness, latency, and cost.

26.10 24.10 `ragopt` migration

Represent each candidate configuration as immutable bindings or references. Submit evaluation plans to `ragjobs`. Preserve current gate decisions and paired comparison semantics through explicit check handlers or deterministic gate artifacts.

The optimizer remains responsible for what constitutes a candidate and acceptable improvement. The job system remains responsible for durable execution and custody.

27 25. Verification strategy

27.1 25.1 Test the laws, not only examples

Example tests catch regressions. Law tests catch abstraction defects.

For `flow`, test:

- worker/batch operational invariance;
- duplicate-key transparency;
- caller-order preservation;
- partial retry;
- cache functionality and corruption;
- budget accounting per attempt;
- alignment custody;
- observer noninterference.

For `ragjobs`, test:

- canonical plan identity;
- sequential associativity and parallel symmetry;
- cycle and port rejection;
- dependency safety;
- lease recovery and stale fencing;
- terminal monotonicity;
- cache reuse and conflict;
- budget reservation;
- concurrency-key exclusion;
- failure propagation modes;
- trigger deduplication;
- restart restoration;
- publication CAS and monotone revision;
- event-prefix integrity.

27.2 25.2 Differential Store testing

The Memory Store is the reference interpreter. A high-value production test generates command sequences and runs them against both Memory and PostgreSQL Stores:

- create;
- claim;
- heartbeat;
- complete;
- fail;
- recover;
- cancel;
- query cache;
- inspect events.

Normalize implementation-specific timestamps and IDs, then compare snapshots, errors, events, cache entries, budgets, and terminal states.

27.3 25.3 Crash injection

Test process termination at every transition boundary:

- before and after run creation commit;
- after claim before handler start;
- during provider call;
- after artifact write before completion;

- after completion commit before acknowledgement;
- after publication commit before completion;
- during child activation;
- during cancellation;
- during outbox relay.

The expected result is not always immediate success. It is a classified, recoverable state with preserved evidence and no invalid publication.

27.4 25.4 Race and schedule tests

Run race-enabled tests, but also permute schedules. Race freedom does not imply semantic invariance.

For pure or fixed-observation handlers, execute fair FIFO, LIFO, randomized, and parallel schedules and compare terminal artifact maps. Event interleavings may differ.

27.5 25.5 Property generation

Generate small acyclic plans and verify:

- only roots are initially available;
- no child runs before dependencies;
- every successful run has all nodes succeeded;
- every terminal run is monotone;
- all event sequences are prefixes;
- all accepted completions had current fences;
- plan finalization is idempotent;
- fragment laws hold for disjoint node names.

27.6 25.6 Production adapter test matrix

Before a PostgreSQL or River profile is authoritative, add:

1. live Store conformance tests;
2. multi-process claim contention;
3. database failover and authoritative-time tests;
4. lease expiry under pauses;
5. transaction rollback at each Store method;
6. cache contention with equal and unequal outputs;
7. River duplicate and transactional completion tests if used;
8. source relay duplicate, reorder, and gap tests;
9. object-store conditional-write and corruption tests;
10. serving alias adoption and rollback tests.

28 26. Operations and observability

28.1 26.1 Durable events versus telemetry

Durable events record semantically important lifecycle transitions. Metrics, logs, and traces provide operational detail.

A run event includes run, optional node, attempt, fence, type, time, worker, class, and bounded data. Fine-grained `flow` events normally belong in tracing or a referenced summary artifact rather than the durable node table.

28.2 26.2 Metrics

Useful bounded-cardinality metrics include:

- runs by plan/state/system/corpus;
- node attempts and failures by handler kind/version/class;
- ready age and queue latency;
- handler duration;
- lease recovery rate;
- consumed and reserved cost;
- cache hit and conflict counts;
- artifact bytes and verification failures;
- source revision lag and oldest pending age;
- gate accept/reject rate;
- alias generation and serving adoption lag.

Do not use run IDs, entity IDs, full digests, or error messages as metric labels.

28.3 26.3 Tracing

A durable run may last longer than a conventional trace root. Use span links across attempts and processes. Include run, node, attempt, fence, semantic key prefix, and artifact reference attributes.

The durable event log remains authoritative after sampled traces expire.

28.4 26.4 Administrative actions

Controls should include:

- cancel run;
- create rerun;
- future derived retry-from-node;
- pause or drain physical queues;
- manual promotion;
- privileged rollback;
- cache invalidation;
- artifact quarantine;
- release quarantine.

Every action records actor, reason, previous state, requested state, and incident/correlation identity.

28.5 26.5 Incident interpretation

28.5.1 Semantic cache conflict

Quarantine the handler kind/version and key. Preserve both outputs. Investigate incomplete identity, moving dependencies, nondeterminism, and corruption. Do not classify as an ordinary transient.

28.5.2 Stuck running node

Inspect lease expiration, worker heartbeat, and Store time. Use the recovery transition; do not manually clear ownership.

28.5.3 Outbox gap

Hold later revisions. Repair or explain the missing revision before advancing the committed cursor.

28.5.4 Gate rejection

Leave the active alias unchanged. Inspect immutable evidence and policy identity. Create a new corrected run rather than editing the decision.

28.5.5 Bad active artifact

Pause automatic publication, perform audited rollback, verify serving adoption, quarantine the candidate, and preserve the incident run.

29 27. Security and multi-tenant concerns

29.1 27.1 Secrets

Plans, keys, events, and artifacts may persist for years. They must not contain credential values. Persist a secret-manager reference or provider profile name and resolve credentials at attempt time.

Changing credentials normally does not alter semantic identity. Changing endpoint, account-specific model behavior, or provider semantics does and must be represented separately.

29.2 27.2 Authorization projection

Authorization should be applied before selective views and generation. A derived artifact must not silently lower the access requirements of its sources.

For multi-tenant systems, include tenant/corpus scope in:

- source stream;
- artifact namespace;
- alias name;
- concurrency key;
- authorization checks;
- cache-sharing policy.

A content digest alone does not prevent one tenant from learning another tenant produced the same value.

29.3 27.3 Immutable artifacts and deletion

Content addressing complicates legal deletion. Production designs need classification metadata, scoped encryption keys, purge workflows, and evidence that all reachable and cached copies are removed.

Cryptographic erasure through tenant/corpus keys can complement physical garbage collection, but it is outside the reference kernel.

29.4 27.4 Supply-chain identity

Workers should report build identity and supported handler protocols. Artifact manifests should record builder and dependency versions. Container images and toolchains should be pinned by digest where operationally required.

30 28. Performance and scaling

30.1 28.1 Scale the right layer

`flow` handles high-cardinality fine-grained operations efficiently inside a node. `ragjobs` handles low-cardinality durable artifact boundaries.

A typical build might use:

```
10--100 durable nodes
10,000--1,000,000 local flow requests
```

Creating a durable row for every embedding request is usually unnecessary unless each request has independent multi-hour cost or external reconciliation requirements.

30.2 28.2 Queue decomposition

Separate worker queues by resource class:

- control/snapshot;
- CPU indexing;
- embedding;
- evaluation;
- publication;
- low-priority backfill.

This prevents a large evaluation from blocking publication or control work. Concurrency keys handle narrower global serialization.

30.3 28.3 Backpressure

Backpressure starts before run creation. When source changes arrive faster than builds complete:

- coalesce contiguous changes more aggressively;
- cap active builds per corpus;
- supersede stale unpublished candidates under explicit policy;
- preserve urgent/manual bypass;
- expose freshness degradation.

Do not create millions of near-identical runs and rely on the queue to recover.

30.4 28.4 Embedding critical path

For m chunks, batch size b, provider concurrency p, and mean request duration t_e:

$$T_e \approx \left\lceil \frac{m}{b} \right\rceil \frac{t_e}{p},$$

subject to request and token quotas. `flow` optimizes this inner term. `ragjobs` determines whether the complete dense artifact can be reused or recovered.

30.5 28.5 Whole build path

A rough critical path is:

$$T_{build} \approx T_{snapshot} + T_{extract} + T_{chunk} + \max(T_{lexical}, T_{dense}) + T_{assemble} + T_{verify} + T_{eval} + T_{publish}.$$

Immutable artifact reuse can reduce individual terms to lookup and verification.

31 29. Limitations and future extensions

31.1 29.1 No live PostgreSQL interpreter in this version

The reference includes schemas and transition guidance, but a production SQL Store still requires implementation and live conformance testing under concurrency, crash, and migration scenarios.

31.2 29.2 No compiled River adapter

The architecture defines River as an optional physical transport. A concrete adapter must select one retry authority and verify transactional dispatch/completion behavior against the installed River version.

31.3 29.3 Nominal ports

Ports validate type names, not artifact payload schemas. A future schema registry can validate JSON Schema, Protocol Buffers descriptors, or generated Go codecs at completion and handler boundaries.

31.4 29.4 Static plans

The current plan is finite and static. Dynamic fan-out can be added by allowing a node to materialize a deterministic validated subplan whose identity depends on enumeration output. This requires explicit limits, stable naming, and replay equality.

31.5 29.5 Partial rerun

The current repair model creates a new run and relies on semantic reuse. A future derived-run operation can explicitly adopt selected successful upstream outputs while retaining lineage to the parent.

31.6 29.6 Richer resource grades

`MaxCost`, queue, priority, and concurrency key are intentionally simple. Future admission may include estimated bytes, memory, GPU, locality, privacy class, or token envelopes.

Such extensions should remain serializable and should not turn plan validation into an opaque static-analysis problem.

31.7 29.7 Segment-level incremental indexing

The immutable full-build baseline can evolve into immutable segments:

```
snapshot delta
  -> normalize changed documents
  -> build lexical/vector segments
  -> assemble manifest over old + new segments + tombstones
  -> verify/evaluate/publish manifest
  -> compact in a separate plan
```

The published value remains an immutable manifest. Compaction creates a new candidate rather than mutating active segments.

31.8 29.8 Formal mechanization

The package laws are tested but not machine-proved. A small TLA+ model would be valuable for lease expiry, duplicate delivery, cancellation, and concurrent completion. Proof assistants could formalize plan algebra and Store refinement after production protocols stabilize.

32 30. Recommended engineering discipline

The design can be summarized in twelve rules.

1. **Name the observation level.** Separate semantic value, view, and trace.
2. **Define equality before identity.** Keys encode substitution permission.
3. **Keep pure computation ordinary.** Use Go functions until an effect boundary exists.
4. **Use content-addressed immutable artifacts.** Do not build in the active namespace.
5. **Persist coarse recoverable boundaries.** Keep high-volume item loops local.
6. **Treat cacheability as an assertion.** Conflicts fail closed.
7. **Separate plan, semantic, and attempt identity.** They answer different questions.
8. **Fence accepted state transitions.** At-least-once attempts are normal.
9. **Make publication small and atomic.** Verify before alias CAS.
10. **Version persisted protocols.** Historical data outlives deployments.
11. **Use explicit narrow graft points.** Applications assemble capabilities.
12. **Test laws and failure boundaries.** Race freedom alone is not semantic correctness.

33 Appendix A. Package map

33.1 A.1 `flow`

```
flow/
  key.go                 semantic request identity
  call.go                keyed typed request
  processor.go           aligned batch effect
  executor.go            batching, workers, cache, retry, order
  result.go              outcomes and operational report
  retry.go               classification and backoff
  admission.go           resource-policy interface
  event.go               bounded operational observation
  cache.go               cache contract and entry validation
  codec.go               typed value encoding
  clock.go               deterministic timing boundary
  cache/
    memory.go
    file.go
  admission/
    budget.go
    rate.go
    chain.go
  examples/embedding/
```

33.2 A.2 `ragjobs`

```
ragjobs/
  ref.go                 semantic references
  canonical.go           canonical JSON support
  artifact.go            immutable descriptors and output digest
```

```
input.go                trigger, values, budget, semantic projection
plan.go                 plan/node schema and finalization
compose.go              optional Atom/Then/Parallel syntax
semantic.go             node invocation keys
state.go                run/node/attempt/event vocabulary
store.go                atomic transition interface
handler.go              versioned protocol and registry
worker.go               transport-neutral lease interpreter
errors.go               failure classification
store/memory/           executable reference semantics
store/file/             restartable local interpreter
storetest/              reusable conformance suite
artifactfs/             immutable local objects
publish/                alias CAS implementations
trigger/                normalized changes and contiguous batches
migrations/postgres/    production schema and helper contract
examples/indexing/      complete durable demonstration
```

33.3 A.3 Integration example

The separate integration module imports both packages and executes `flow` inside a `ragjobs` handler. This proves that the dependency direction requires no coupling between the kernels.

34 Appendix B. Transition table

Transition	Required state	Principal checks	Result
CREATE	run absent	plan valid; dedup absent	queued run; roots available
CLAIM	node available	time, queue, budget, key	running; attempt/fence incremented
HEARTBEAT	current live lease	owner, attempt, fence, expiration	lease extended
CACHE REUSE	current live lease	semantic key present	ordinary reused completion
COMPLETE	current live lease	outputs, digest, cost, cache consistency	succeeded; descendants propagated
RETRY	current running lease	retryable; attempts/budget/deadline remain	available at durable retry time
FAIL	current running lease	terminal condition	failed; cancel or skip propagation
RECOVER	expired running lease	expiration reached	lease-expired attempt; retry/fail
CANCEL	run nonterminal	authenticated reason	remaining work and run canceled
FINALIZE	all nodes terminal	outcome classification	run succeeded/failed/canceled

35 Appendix C. Law checklist

35.1 C.1 `flow`

```
KeyComplete:
  equal key => substitution is permitted

DuplicateTransparent:
  same key appears many times => one unique execution outcome

OrderPreserved:
  outcome[i].key == call[i].key

OperationalInvariant:
  workers/batches do not change semantic values under declared assumptions

PartialRetry:
  successful items do not re-enter later waves

CacheFunctional:
  one key maps to at most one successful byte value

AdmissionCountsAttempts:
  every physical batch attempt is admitted and charged

Alignment:
  successful processor return count equals input count
```

35.2 C.2 `ragjobs`

```
PlanCanonical:
  irrelevant ordering/default representation yields one plan ID

ThenAssociative:
  (P;Q);R == P;(Q;R)

ParallelSymmetric:
  P||Q == Q||P modulo canonical ordering

DependencySafe:
  claim(node) => all direct dependencies succeeded

FenceMonotone:
  every later claim has a greater fence

StaleExcluded:
  old lease cannot complete after ownership changes

TerminalMonotone:
  terminal state never reopens

CacheFunctional:
  one semantic key maps to at most one output digest

BudgetSafe:
```

```
  consumed + reserved never exceeds positive maximum under honest bounds

ConcurrencyExclusive:
  at most one running owner per nonempty concurrency key

EventPrefix:
  one run has events 1..N without gaps

PublicationCAS:
  active alias changes only under expected state and revision policy
```

36 Appendix D. Production readiness checklist

Automatic publication should remain disabled until:

- the durable Store passes conformance and crash tests;
- source change capture proves no-loss, deduplication, and gap behavior;
- every handler has a protocol version, timeout, retry classification, cache decision, and cost bound;
- every artifact is immutable and independently verifiable;
- semantic cache conflicts are alerted and quarantined;
- quality policy and baseline resolution are immutable and versioned;
- alias compare-and-swap, idempotent replay, stale revision rejection, manual promotion, and rollback are tested;
- serving processes report loaded generation and health;
- cancellation and lease recovery have been exercised;
- secrets and tenant boundaries have been reviewed;
- retention, garbage collection, backup, and disaster recovery have runbooks;
- shadow runs show acceptable semantic equivalence, latency, and cost.

37 Appendix E. Glossary

Admitter. Local `flow` policy that approves or rejects one physical batch attempt under budgets or rate limits.

Artifact. Immutable validated value descriptor with semantic `Ref` and custody metadata.

Attempt. One physical ownership interval for a durable node.

Cacheability. Assertion that one semantic key has one successful output equality class.

Candidate. Immutable build output not yet exposed through the active publication alias.

Canonicalization. Versioned conversion from a semantic value to stable bytes for identity.

Concurrency key. Operational name permitting at most one running durable node globally in a conforming Store.

Control plane. Mutable transactional run, node, lease, cache-index, trigger, event, and alias state.

Data plane. Immutable snapshots, corpora, indexes, bundles, and evidence.

Deduplication key. Operational identity suppressing duplicate run creation.

Fence. Monotonically increasing token required for current ownership transitions.

Graft point. Narrow interface where an external implementation is explicitly attached to the kernel.

Handler. Versioned implementation of one coarse durable node protocol.

Lease. Time-bounded fenced capability for one node attempt.

Plan. Canonical immutable versioned dependency graph.

Processor. Aligned batch effect interpreted by `flow`.

Publication. Conditional transition from a mutable alias to an immutable artifact reference.

Reference. Kind- and version-qualified semantic content identity.

Semantic key. Digest defining when one successful node or request result may substitute for another.

Trace. Operational history, distinct from semantic value.

View. Deterministic selective projection of a fixed semantic state under a versioned policy.

38 Appendix F. Prior work and implementation evidence

The design and terminology build on the supplied project sources and the following project documents:

- *rag-ttc Semantic Architecture Handbook: Turning a reproducible RAG toolbox into a composable evidence system with rigorous semantics*, August 3, 2026.
- *rag-ttc Composition Pass Playbook: Second-pass integration and evaluation protocol*, August 4, 2026.
- *Compositional Durable Orchestration for Production Retrieval-Augmented Generation: A Denotational and Operational Semantics for Change-Driven Indexing, Evaluation, and Publication in TTC and GEC*, August 7, 2026.
- the current `rag-ttc`, `ragkit`, `ragopt`, TTC, and GEC/CoinVault source snapshot and optimization artifacts.

The standalone implementation in the companion source distribution is the evidence for API and behavior claims in this handbook. The prior documents are architectural sources, not substitutes for executable verification.

39 Closing perspective

The central result is not a new universal workflow framework. It is a sharper division of responsibility.

`flow` interprets many semantically identified requests efficiently inside one attempt. `ragjobs` interprets a canonical durable graph across attempts and failures. Immutable artifacts connect the layers. A small alias transition connects candidate construction to serving. Explicit references connect optimization, evaluation, and production without forcing them into one ontology.

The elegance comes from refusing to make one abstraction own every concern. The semantic soundness comes from giving each boundary a precise equality, identity, transition contract, and conformance test.